Advanced Quantum Computing

Mathematics, algorithms and research for Athena

This book bridges introductory quantum computing and the mathematical work needed to derive algorithms. Its destination is a research prototype for Athena, a multi-agent assistant that allocates tasks, models resources and plans workflows. All explanations, examples, diagrams and exercises here are newly written; references identify the underlying results.

What to bring. Familiarity with algebra, functions and basic probability is enough to start. Complex linear algebra is developed first. Derivatives and integrals are introduced where needed; full differential-equation theory is not assumed. Paper, a pencil and patience are sufficient for the worked examples. Code is explanatory pseudocode and does not require a laptop to follow.

How to study. Read one numbered lesson at a time. Before looking at a worked calculation, attempt the next line yourself. Reproduce the derivation with different numbers. A useful mastery test is to explain why each conjugate, normalisation factor and sign is necessary. Chapter 9 provides answer checks, a solution workshop and a research protocol. Added derivation drills connect the algebra to array indexing, circuit synthesis and QUBO compilation.

Scope. Forty pages cannot replace a semester of mechanics, complexity theory and optimisation. The core derivations receive priority. Error correction, cryptographic security proofs and number-theoretic success bounds are introduced with clear limits; the references provide the next course of study. No claimed improvement to Athena is an experimental result.

Three questions to carry through the book
What mathematical object is being transformed? What can actually be measured? What does the proposed speedup count, and what costs does it omit?

A4 portrait / 11-point body / monochrome vector diagrams

Self-contained lessons, original calculations, exercises with checks, and a reproducible optimisation design.

Contents

1 Mathematical foundations **3**

2 Mathematical quantum mechanics **7**

3 Quantum computation **11**

4 Quantum algorithms **15**

5 Quantum optimisation **22**

6 Quantum machine learning **27**

7 Quantum cryptography **31**

8 Athena: advanced algorithm research **34**

9 Exercises and research challenges **38**

References and further study **40**

Notation and conventions

Symbol	Meaning used throughout
i, z^*, $A^\dagger$	$\mathrm{i}^2 = -1$; complex conjugate; conjugate transpose.
$\lvert\psi\rangle$, $\langle\psi\rvert$	Column vector and its conjugate-transpose row.
$\langle\phi \mid \psi\rangle$	Inner product, conjugate-linear in the first argument.
I, $\otimes$, Tr	Identity matrix, tensor product, sum of diagonal entries.
H, H	Hamiltonian (energy operator); Hadamard gate, respectively.
n, $N = 2^n$	Number of qubits; size of the computational basis.
X, Y, Z	Pauli matrices; subscripts identify the qubit acted on.
$\hbar$, t, θ	Reduced Planck constant, time, and an angle in radians.
$O(g(n))$, $\Theta(g(n))$	Asymptotic upper bound; matching upper and lower order.
x, s, Q	Binary variables, spins in $\{-1, +1\}$, upper-triangular QUBO.

Register order. In $\lvert q_0 q_1 \cdots q_{n-1}\rangle$, the leftmost bit is most significant. Two-qubit vectors are ordered $00, 01, 10, 11$. Circuit time flows left to right; matrix products act right to left. Software packages may display bit strings differently: translate the order before comparing outputs.

Evidence labels. *Established result* means a mathematical statement with stated assumptions. *Existing algorithm* identifies a known procedure. *Proposed application* means an Athena engineering design. *Hypothesis* means a claim that needs experiments. Square-bracket citations refer to the bibliography, not to required external reading.

1 Mathematical foundations

1.1 Complex numbers and phase

Source trail: [1], *Lesson 1;* [2], *Lecture 3;* [3], *Chapter 2. References on p. 40.*

A classical probability is nonnegative, but a quantum amplitude must also encode phase. Complex numbers supply two real coordinates while supporting addition, multiplication and a length compatible with interference. Write $z = a + \mathrm{i}b$, where $a, b \in \mathbb{R}$. Its conjugate is $z^* = a - \mathrm{i}b$, and

$$|z|^2 = z^* z = a^2 + b^2, \qquad z = re^{\mathrm{i}\theta}, \quad r = |z| \geq 0. \tag{1.1}$$

The angle θ is measured from the positive real axis. Multiplying two polar forms multiplies their lengths and adds their angles. This is why complex multiplication describes phase evolution so naturally.

Euler's formula $e^{\mathrm{i}\theta} = \cos\theta + \mathrm{i}\sin\theta$ follows by grouping the even and odd powers of the exponential series. For a real or complex number w, this series is $e^w = \sum_{k=0}^{\infty} w^k/k!$, with $k! = 1 \cdot 2 \cdots k$ and $0! = 1$. Because $\mathrm{i}^{2k} = (-1)^k$ and $\mathrm{i}^{2k+1} = \mathrm{i}(-1)^k$, the two groups are the cosine and sine series. Setting $\theta = \pi$ gives Euler's identity $e^{\mathrm{i}\pi} + 1 = 0$.

Worked example: Adding before squaring

Let two contributions to one output amplitude be $u = 1/2$ and $v = \mathrm{i}/2$. Then $|u+v|^2 = |(1+\mathrm{i})/2|^2 = 1/2$. If instead $v = -1/2$, the result is zero. The magnitudes of the separate contributions are unchanged, but their relative phase changes the answer.

In general,

$$|u + v|^2 = |u|^2 + |v|^2 + 2\,\mathrm{Re}(u^* v). \tag{1.2}$$

The last term is interference. Re extracts the real part. It may be positive, zero or negative; the complete squared modulus is always nonnegative. Contributions to the *same* outcome add as amplitudes. Probabilities of distinct, mutually exclusive outcomes add as probabilities.

A state vector can be multiplied by a common phase $e^{\mathrm{i}\gamma}$ without changing any probabilities. Indeed, $|e^{\mathrm{i}\gamma} z|^2 = |z|^2$. Relative phases between components survive this cancellation and are observable after a suitable change of basis. Do not throw them away during a calculation.

Paper check. Find $(1 + \mathrm{i})(2 - \mathrm{i})$ and $|(1 + \mathrm{i})/2|$. Answers: $3 + \mathrm{i}$ and $1/\sqrt{2}$. Explain why squaring $(1 + \mathrm{i})/2$ itself does not produce a probability.

Derivation drill: normalise before measuring. For $v = (1+\mathrm{i}, 2)^T$, compute $v^\dagger v = (1-\mathrm{i})(1+\mathrm{i}) + 4 = 6$, so $|\psi\rangle = v/\sqrt{6}$. The probabilities are $2/6$ and $4/6$. A phase i multiplying the entire vector changes neither. By contrast, changing only its second coordinate changes its overlap with $|+\rangle$. For the original vector, $\langle + \mid \psi\rangle = (3 + \mathrm{i})/\sqrt{12}$, so $p(+) = 10/12 = 5/6$.

Try it first. Replace 2 by -2. The norm stays $\sqrt{6}$, but $p(+) = |-1+\mathrm{i}|^2/12 = 1/6$. The same computational-basis probabilities can conceal very different interference. This is why a simulator must store complex amplitudes until measurement.

1.2 Vectors, matrices and linear maps

A vector in $\mathbb{C}^d$ is an ordered list of d complex coordinates. Addition and scalar multiplication operate coordinate by coordinate. A vector space is a collection closed under these operations and satisfying their familiar algebraic rules. A basis is a linearly independent set that spans the space: every vector has a unique expansion in it. The number of basis vectors is the dimension.

For one qubit, the computational basis is $e_0 = (1,0)^T$ and $e_1 = (0,1)^T$, where T denotes transpose. A column $v = (\alpha, \beta)^T$ equals $\alpha e_0 + \beta e_1$. Quantum states impose an additional length constraint, so the set of normalised states alone is not a vector space: adding two unit vectors need not give another unit vector.

A linear map obeys $A(au + bv) = aAu + bAv$ for all scalars a, b. A matrix represents such a map after choosing bases. For $A \in \mathbb{C}^{m\times d}$ and $v \in \mathbb{C}^d$,

$$(Av)_j = \sum_{k=0}^{d-1} A_{jk} v_k. \tag{1.3}$$

The kth column is the image of e_k. This makes a gate easy to specify: give its action on every basis vector and extend linearly. An arbitrary rule on only some states is not yet a valid gate.

Worked example: Matrix multiplication with complex entries

Take $A = \begin{pmatrix} 1 & \mathrm{i} \\ 0 & 2 \end{pmatrix}$ and $v = \begin{pmatrix} 1 \\ \mathrm{i} \end{pmatrix}$. Then

$$Av = \begin{pmatrix} 1 + \mathrm{i}^2 \\ 2\mathrm{i} \end{pmatrix} = \begin{pmatrix} 0 \\ 2\mathrm{i} \end{pmatrix}.$$

The zero in the first component comes from cancellation. This A is a useful linear map but is not a valid isolated-system quantum evolution: it does not preserve lengths.

If $B \in \mathbb{C}^{d\times r}$, composition is represented by AB, with $(AB)_{jk} = \sum_\ell A_{j\ell} B_{\ell k}$. The inner dimensions must match. The rightmost operation occurs first. Usually $AB \neq BA$, and reversing gates can change measured probabilities.

An inverse A^{-1} reverses a square map when it exists: $A^{-1}A = I$. The identity I leaves all vectors unchanged. Not every matrix has an inverse; a map that sends two independent input directions to one direction loses information. Reversible quantum evolution must have an inverse, which will take a particularly useful form on the next page.

Paper check. For $X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ and $Z = \mathrm{diag}(1, -1)$, calculate XZe_0 and ZXe_0. You should obtain e_1 and $-e_1$. Compare also their action on e_1.

From columns to code. For H, compute the composition rather than inferring it from one input:

$$\mathsf{H}^2 = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = \frac{1}{2}\begin{pmatrix} 2 & 0 \\ 0 & 2 \end{pmatrix} = I.$$

The top-right zero is $1\cdot 1 + 1\cdot(-1)$; the lower-right two is $1\cdot 1 + (-1)(-1)$. Implement a matrix product with three indices: output row, output column, and the summed inner index. Initialising each output sum to zero avoids carrying terms from another coordinate.

Exercise with a diagnostic. Check the earlier nonunitary A on v: $\|v\|^2 = 2$ but $\|Av\|^2 = 4$. Normalising its output afterwards does not turn A into a deterministic quantum gate: the required scaling would depend on the input and destroy linearity.

1.3 Inner products, spectra and unitary matrices

The inner product generalises a dot product while retaining nonnegative squared length:

$$\langle u, v\rangle = u^\dagger v = \sum_j u_j^* v_j, \qquad \|v\| = \sqrt{v^\dagger v}. \tag{1.4}$$

The dagger means transpose *and* conjugate. Orthogonality means $u^\dagger v = 0$. An orthonormal basis has unit-length vectors with zero pairwise inner products. Projection onto a unit vector u is $uu^\dagger v$; the scalar $u^\dagger v$ is its coordinate along that direction. The Cauchy–Schwarz inequality $|u^\dagger v| \le \|u\|\|v\|$ ensures overlaps of unit states never exceed one.

An eigenvector $v \neq 0$ obeys $Av = \lambda v$ for an eigenvalue λ: the map changes only its scale or phase. Eigenvalues solve $\det(A-\lambda I) = 0$. For a 2×2 matrix $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$, the determinant is $ad-bc$; it vanishes when the columns are dependent. A Hermitian matrix satisfies $A = A^\dagger$. Its eigenvalues are real: the scalar $v^\dagger Av$ equals its own conjugate since $A = A^\dagger$. Hence $\lambda = (v^\dagger Av)/(v^\dagger v)$ is real, because $v^\dagger v > 0$. Finite-dimensional Hermitian matrices admit an orthonormal eigenbasis, giving the spectral decomposition

$$A = \sum_j \lambda_j u_j u_j^\dagger. \tag{1.5}$$

This theorem makes observables, Hamiltonians and matrix exponentials tractable.

Worked example: Diagonalising a non-diagonal observable

Start with $Xv = \lambda v$ and $v = (a, b)^T \neq 0$. Moving all terms left gives

$$\begin{pmatrix} -\lambda & 1 \\ 1 & -\lambda \end{pmatrix}\begin{pmatrix} a \\ b \end{pmatrix} = 0, \qquad \det(X - \lambda I) = (-\lambda)^2 - 1 = 0.$$

Thus $\lambda = \pm 1$. For $+1$, $-a + b = 0$, so choose $(a, b) = (1, 1)$. Its squared norm is 2; divide by $\sqrt{2}$. For -1, $a + b = 0$, so choose $(1, -1)$ and normalise similarly. Therefore $u_\pm = (1, \pm 1)^T/\sqrt{2}$. Form the two outer products:

$$u_+ u_+^\dagger = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad u_- u_-^\dagger = \frac{1}{2}\begin{pmatrix} 1 & -1 \\ -1 & 1 \end{pmatrix}.$$

Their difference is X. This reconstructs the original matrix and checks both eigenvalues and normalisation.

A unitary matrix obeys $U^\dagger U = UU^\dagger = I$. Consequently $\|Uv\|^2 = v^\dagger U^\dagger Uv = \|v\|^2$, and $U^{-1} = U^\dagger$. All inner products are preserved, not just lengths. Its eigenvalues have modulus one: if $Uv = \lambda v$, length preservation implies $|\lambda| = 1$.

Hermitian and unitary are different properties. X happens to satisfy both, while $\mathrm{diag}(2, 3)$ is Hermitian but not unitary. The phase matrix $\mathrm{diag}(1, \mathrm{i})$ is unitary but not Hermitian. A Hermitian matrix describes measurable values; a unitary describes a reversible transformation. Confusing them obscures the distinction between energy and time evolution.

Paper check. Verify $(1, \mathrm{i})^T/\sqrt{2}$ is orthogonal to $(1, -\mathrm{i})^T/\sqrt{2}$. Omitting conjugation gives the wrong answer.

Checking a unitary by columns. Set $U = (u_+, u_-) = \mathsf{H}$. Its diagonal Gram entries are $u_\pm^\dagger u_\pm = 1$, and its off-diagonal entries are $(1 - 1)/2 = 0$. Hence $U^\dagger U = I$ and $U^\dagger X U = \mathrm{diag}(1, -1)$. The matrix U changes eigenbasis coordinates into computational coordinates; $U^\dagger$ reverses that change. For any proposed eigenpair, compute the residual $Av - \lambda v$; it must vanish. For a numerical implementation, compare its norm with a tolerance rather than demanding exact floating-point equality.

1.4 Tensor products and amplitude bookkeeping

Two systems combine by a tensor product. If one space has dimension d and another r, the joint space has dimension dr. For column vectors $u = (u_0, u_1)^T$ and $v = (v_0, v_1)^T$,

$$u \otimes v = (u_0 v_0, u_0 v_1, u_1 v_0, u_1 v_1)^T. \tag{1.6}$$

This ordering places the first system in the more significant bit. For matrices, replace each entry A_{jk} by the block $A_{jk}B$ to form $A \otimes B$. Local transformations satisfy $(A \otimes B)(u \otimes v) = (Au) \otimes (Bv)$.

Worked example: A complex two-qubit product state

Let $u = (1, \mathrm{i})^T/\sqrt{2}$ and $v = (\sqrt{3}, 1)^T/2$. Expand before collecting coordinates:

$$\begin{aligned} u \otimes v &= \frac{(|0\rangle + \mathrm{i}|1\rangle) \otimes (\sqrt{3}|0\rangle + |1\rangle)}{2\sqrt{2}} \\ &= \frac{\sqrt{3}|00\rangle + |01\rangle + \mathrm{i}\sqrt{3}|10\rangle + \mathrm{i}|11\rangle}{2\sqrt{2}}. \end{aligned}$$

Thus $\psi = (\sqrt{3}, 1, \mathrm{i}\sqrt{3}, \mathrm{i})^T/(2\sqrt{2})$. For example, $p(10) = |\mathrm{i}\sqrt{3}/(2\sqrt{2})|^2 = 3/8$; conjugation removes the imaginary phase. The probabilities for $00, 01, 10, 11$ are $3/8, 1/8, 3/8, 1/8$, summing to one. Summing the first two gives probability $1/2$ for first bit zero. Summing entries with second bit zero gives $3/4$. These are the separate input probabilities, as a product preparation predicts.

For n qubits, write $|\psi\rangle = \sum_{x=0}^{2^n-1} \alpha_x |x\rangle$. The symbols $|x\rangle$ are computational-basis columns. Normalisation is $\sum_x |\alpha_x|^2 = 1$, and $|\alpha_x|^2$ is the probability of basis outcome x. Storing a general state explicitly requires 2^n complex numbers on a classical computer. This is a representation cost, not a proof that a quantum processor can return all those numbers.

Not every joint vector factors. For two qubits with amplitudes a, b, c, d, a nonzero vector is a product exactly when

$$ad - bc = 0. \tag{1.7}$$

To see why, arrange the amplitudes in $M = \begin{pmatrix} a & b \\ c & d \end{pmatrix}$. A product vector makes $M = uv^T$, a rank-one matrix (one independent column direction). Conversely, a nonzero 2×2 matrix with zero determinant has rank one and therefore factors. The Bell vector $(1, 0, 0, 1)^T/\sqrt{2}$ has determinant $1/2$ and is entangled.

Tensor products also preserve inner products multiplicatively: $\langle u \otimes v, w \otimes z\rangle = \langle u, w\rangle\langle v, z\rangle$. Use this to check normalisation without expanding every coordinate. For operators, $(A \otimes B)^\dagger = A^\dagger \otimes B^\dagger$.

Paper check. Expand $(e_0 - e_1) \otimes (e_0 + \mathrm{i}e_1)/2$. Is it entangled? Check $ad - bc$ before answering.

How a local matrix becomes a joint operator. Block expansion gives

$$\mathsf{H} \otimes I = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{pmatrix}.$$

It mixes amplitudes $(00, 10)$ and $(01, 11)$, because the first bit changes and the second stays fixed. Conversely, $I \otimes \mathsf{H}$ mixes $(00, 01)$ and $(10, 11)$. This is the indexing rule behind the pair-update simulator in Chapter 3.

Solved tensor exercise. Distributing the paper-check expression gives $(1, \mathrm{i}, -1, -\mathrm{i})^T/2$. Thus $ad - bc = (-\mathrm{i}/4) - (-\mathrm{i}/4) = 0$. Recover its factors from rows: the second row is minus the first, so it is $(1, -1)^T/\sqrt{2} \otimes (1, \mathrm{i})^T/\sqrt{2}$. Always check both the factorisation and the norm.

2 Mathematical quantum mechanics

2.1 Hilbert spaces and wavefunctions

Source trail: [2], *Lecture 3;* [3], *Chapters 2–3. References on p. 40.*

A Hilbert space is an inner-product vector space complete in its norm: every sequence whose elements become arbitrarily close has a limit inside the space. Finite-dimensional $\mathbb{C}^d$ is automatically complete. Infinite-dimensional spaces need this condition so that limits of state approximations remain meaningful.

Dirac notation packages linear algebra. A ket $|\psi\rangle$ is a column vector; its bra $\langle\psi|$ is the conjugate transpose. The inner product $\langle\phi \mid \psi\rangle$ is a scalar. The outer product $|\psi\rangle\langle\phi|$ is an operator: it sends $|v\rangle$ to $|\psi\rangle\langle\phi \mid v\rangle$. In an orthonormal basis,

$$I = \sum_j |j\rangle\langle j|, \qquad |\psi\rangle = \sum_j \langle j \mid \psi\rangle |j\rangle. \tag{2.1}$$

The first identity expresses completeness. Inserting it between operators converts abstract expressions into coordinate calculations.

A spinless particle on a line instead has a position wavefunction $\psi(x) = \langle x|\psi\rangle$, where x is continuous. Its space is $L^2(\mathbb{R})$, consisting of square-integrable functions, identified if they differ only on a set of zero measure. Position kets are generalised objects, not normalisable vectors in this space. Here

$$\int_{-\infty}^{\infty} |\psi(x)|^2 \mathrm{d}x = 1, \qquad \Pr(x \in [a,b]) = \int_a^b |\psi(x)|^2 \mathrm{d}x. \tag{2.2}$$

The integral sums density over small intervals of width $\mathrm{d}x$. Thus $|\psi(x)|^2$ is a probability density, not the probability of one exact position.

Worked example: Normalising a continuous state

For $x \geq 0$ let $\psi(x) = Ae^{-x/\ell}$, with length $\ell > 0$, and put $\psi(x) = 0$ otherwise. Compute

$$1 = |A|^2 \int_0^\infty e^{-2x/\ell} \mathrm{d}x = |A|^2 \left[-\frac{\ell}{2} e^{-2x/\ell} \right]_0^\infty = |A|^2 \ell/2.$$

Choosing A real and positive yields $A = \sqrt{2/\ell}$ and $\Pr(0 \leq x \leq \ell) = 1 - e^{-2} \approx 0.865$.

A continuous wavefunction generally needs infinitely many coordinates, whereas a qubit needs two. A physical qubit is a selected two-level subsystem or subspace of a larger physical system. Leakage outside that space violates the ideal qubit model. Discretising a wavefunction on a computer introduces an approximation and an encoding; a numerical grid is not automatically a collection of independent physical qubits.

Check. Expand $|0\rangle\langle 1|$ as a 2×2 matrix. Its only nonzero entry is the top-right entry. Apply it to $\alpha|0\rangle + \beta|1\rangle$ to obtain $\beta|0\rangle$.

Changing coordinates without changing the state. In the basis $|\pm\rangle$, a qubit has coordinates $c_+ = \langle + \mid \psi\rangle$ and $c_- = \langle - \mid \psi\rangle$. For $|\psi\rangle = (\sqrt{3}|0\rangle + \mathrm{i}|1\rangle)/2$, these are $(\sqrt{3} \pm \mathrm{i})/(2\sqrt{2})$. Reconstruct it:

$$c_+|+\rangle + c_-|-\rangle = \frac{c_+ + c_-}{\sqrt{2}}|0\rangle + \frac{c_+ - c_-}{\sqrt{2}}|1\rangle = \frac{\sqrt{3}}{2}|0\rangle + \frac{\mathrm{i}}{2}|1\rangle.$$

A coordinate change is not automatically a physical operation. In code, distinguish expressing the same ket in another basis from applying a gate to a register whose measurement basis stays fixed.

2.2 Hamiltonians and the Schrödinger equation

The Hamiltonian H represents energy and generates time evolution of an isolated system. Continuous norm-preserving evolution motivates the mathematical form; identifying its generator with physical energy is a postulate, not a derivation from algebra alone.

Assume a differentiable unitary evolution with $U(0) = I$. For a small time δt, write $U(\delta t) = I + K\delta t + O(\delta t^2)$. Substituting in $U^\dagger U = I$ gives $K^\dagger + K = 0$. Consequently $K = -\mathrm{i}H/\hbar$ for a Hermitian operator H. Taking the derivative of the state gives

$$\mathrm{i}\hbar\frac{\partial}{\partial t}|\psi(t)\rangle = H|\psi(t)\rangle. \tag{2.3}$$

Here t is time and $\hbar$ is the reduced Planck constant. A derivative measures instantaneous change; when the ket depends only on t, $\mathrm{d}/\mathrm{d}t$ and $\partial/\partial t$ are equivalent. Energy times time divided by $\hbar$ is dimensionless, as a phase must be.

For constant H, its orthonormal energy eigenvectors satisfy $H|E_j\rangle = E_j|E_j\rangle$, and

$$U(t) = e^{-\mathrm{i}Ht/\hbar} = \sum_j e^{-\mathrm{i}E_j t/\hbar}|E_j\rangle\langle E_j|. \tag{2.4}$$

Differentiating verifies Eq. (2.3). Every eigencomponent acquires a phase of modulus one; therefore U is unitary. If $H(t)$ changes with time, the differential equation remains valid, but exponentiating its integral without time ordering generally fails unless Hamiltonians at different times commute, meaning $H(t_1)H(t_2) = H(t_2)H(t_1)$.

Worked example: An energy splitting becomes interference

For $H = \hbar\omega Z/2$, with $Z = \mathrm{diag}(1, -1)$ and frequency ω, start in $(|0\rangle + |1\rangle)/\sqrt{2}$. At time t,

$$|\psi(t)\rangle = \frac{e^{-\mathrm{i}\omega t/2}|0\rangle + e^{\mathrm{i}\omega t/2}|1\rangle}{\sqrt{2}}.$$

Taking the bra of the initial state gives

$$\langle + \mid \psi(t)\rangle = \frac{e^{-\mathrm{i}\omega t/2} + e^{\mathrm{i}\omega t/2}}{2} = \cos(\omega t/2).$$

Measuring in the plus/minus basis gives $\Pr(+) = \cos^2(\omega t/2)$, although computational-basis probabilities stay uniform.

For a particle of mass m in potential $V(x)$, the position equation is $\mathrm{i}\hbar\partial_t\psi = [-\hbar^2\partial_x^2/(2m) + V(x)]\psi$. Spatial curvature, the second derivative, represents kinetic energy. This is a differential operator requiring boundary and domain conditions, unlike a qubit's finite matrix.

Check. Adding cI to a Hamiltonian gives the global phase $e^{-\mathrm{i}ct/\hbar}$. Explain why probabilities are unchanged.

Check the differential equation component by component. Expand $|\psi(t)\rangle = \sum_j c_j(t)|E_j\rangle$ in a fixed energy basis. Multiplying Eq. (2.3) by $\langle E_j|$ gives $\mathrm{i}\hbar\dot{c}_j = E_j c_j$. The solution $c_j(t) = c_j(0)e^{-\mathrm{i}E_j t/\hbar}$ follows by differentiating the exponential: $\dot{c}_j = (-\mathrm{i}E_j/\hbar)c_j$. Multiplication by $\mathrm{i}\hbar$ returns $E_j c_j$, checking the sign and units. For implementation, diagonalise once when H is fixed, evolve these scalar phases, then transform back.

2.3 Observables, measurement and the Born rule

An observable A is Hermitian. Its distinct eigenvalues a are possible outcomes. Let P_a project onto the corresponding eigenspace. Then $A = \sum_a a P_a$, $\sum_a P_a = I$, and $P_a P_b = 0$ for distinct outcomes. The projective measurement postulate is

$$p(a) = \langle\psi|P_a|\psi\rangle, \qquad |\psi_a\rangle = P_a|\psi\rangle/\sqrt{p(a)} \quad (p(a) > 0). \tag{2.5}$$

If $P_a = |a\rangle\langle a|$, the probability reduces to $|\langle a \mid \psi\rangle|^2$, the Born rule. A degenerate eigenvalue has a larger eigenspace; the updated state can retain superposition within it.

Repeated preparations give an expectation and variance,

$$\mathbb{E}[A] = \sum_a a\, p(a) = \langle\psi|A|\psi\rangle, \quad \mathrm{Var}(A) = \langle\psi|A^2|\psi\rangle - \mathbb{E}[A]^2. \tag{2.6}$$

An expectation is an average, not necessarily a possible single outcome. Variational algorithms estimate such averages by repeating circuits.

Worked example: A complex state in different bases

Take $|\psi\rangle = (\sqrt{3}|0\rangle + \mathrm{i}|1\rangle)/2$. In the Z basis its probabilities are $3/4$ and $1/4$, so $\mathbb{E}[Z] = 1/2$. The X eigenstates are $|\pm\rangle = (|0\rangle \pm |1\rangle)/\sqrt{2}$, and

$$\langle \pm \mid \psi\rangle = \frac{\sqrt{3} \pm \mathrm{i}}{2\sqrt{2}}.$$

For example, $p(+) = (\sqrt{3} - \mathrm{i})(\sqrt{3} + \mathrm{i})/8 = (3+1)/8 = 1/2$; the minus outcome has the same value. After outcome $+$, the state is $|+\rangle$; an immediate repeated X measurement gives $+$ with certainty. Since $Z^2 = I$, the initial Z variance is $1 - 1/4 = 3/4$.

General measurements use positive operators E_m summing to I, with $p(m) = \langle\psi|E_m|\psi\rangle$. Positivity means $v^\dagger E_m v \geq 0$ for every v. This collection is called a POVM. It determines probabilities, but a state-update rule requires more information. Operators M_m satisfying $E_m = M_m^\dagger M_m$ describe one implementation; others can disturb states differently.

To measure only the first qubit of a pair, use $P_0 = |0\rangle\langle 0| \otimes I$ and $P_1 = |1\rangle\langle 1| \otimes I$. First compute each projected vector's squared norm, then normalise the branch you observed. Renormalising branches before computing their probabilities destroys the relative weights.

Check. For $(|00\rangle + \mathrm{i}|11\rangle)/\sqrt{2}$, the first bit is uniform. Conditional on its value being one, the full state is $\mathrm{i}|11\rangle$, physically equivalent to $|11\rangle$.

Solved conditional-state calculation. Let $|\eta\rangle = (|00\rangle + |01\rangle + \sqrt{2}|11\rangle)/2$. Measuring the first bit gives

$$P_0|\eta\rangle = (|00\rangle + |01\rangle)/2, \quad p(0) = 1/4 + 1/4 = 1/2.$$

Dividing by $\sqrt{1/2}$ gives $|\eta_0\rangle = (|00\rangle + |01\rangle)/\sqrt{2} = |0\rangle|+\rangle$. The other branch has probability $1/2$ and becomes $|11\rangle$. A simulator should first sample these branch probabilities, then divide the selected projected vector by its norm. The sum $p(0) + p(1) = 1$ and the updated norm one are separate consistency checks.

2.4 Density matrices, mixtures and noise

If $|\psi_k\rangle$ is prepared with classical probability p_k, the density matrix is

$$\rho = \sum_k p_k |\psi_k\rangle\langle\psi_k|, \qquad \rho = \rho^\dagger, \quad \rho \succeq 0, \quad \operatorname{Tr}\rho = 1. \tag{2.7}$$

Here $\sum_k p_k = 1$, and $\succeq 0$ denotes positivity. Distinct preparation ensembles may yield the same density matrix and identical statistics. A pure state has $\rho = |\psi\rangle\langle\psi|$ and purity $\operatorname{Tr}\rho^2 = 1$; a mixed state has purity below one. In dimension d it is at least $1/d$, obtained by applying Cauchy–Schwarz to the nonnegative eigenvalues.

Worked example: Coherence versus classical uncertainty

A coherent $|+\rangle$ state and a fair mixture of $|0\rangle, |1\rangle$ have matrices

$$\rho_+ = \frac{1}{2}\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix}, \quad \rho_{\text{mix}} = \frac{1}{2}\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Both give uniform Z outcomes, but $\operatorname{Tr}(X\rho_+) = 1$ and $\operatorname{Tr}(X\rho_{\text{mix}}) = 0$. Their purities are 1 and $1/2$. Off-diagonal entries record coherence in the chosen basis.

The general rules are $p(m) = \operatorname{Tr}(E_m\rho)$, $\mathbb{E}[A] = \operatorname{Tr}(A\rho)$, and $\rho' = U\rho U^\dagger$. Projective outcome a updates to $P_a\rho P_a/p(a)$. Discarding the outcome gives $\sum_a P_a\rho P_a$ instead.

Discarding subsystem B uses the partial trace,

$$\rho_A = \operatorname{Tr}_B \rho_{AB} = \sum_b (I \otimes \langle b|)\rho_{AB}(I \otimes |b\rangle), \tag{2.8}$$

where $\{|b\rangle\}$ is an orthonormal basis. The pure Bell state reduces to $I/2$: cross terms vanish because $\langle 0 \mid 1\rangle = 0$. Thus local mixedness need not come from a classical lottery over global states.

A noise channel can be written $\mathcal{E}(\rho) = \sum_k K_k\rho K_k^\dagger$, with $\sum_k K_k^\dagger K_k = I$ to preserve trace. A phase flip with probability p gives $(1-p)\rho + pZ\rho Z$; at $p = 1/2$ it erases the off-diagonal entries. Quantum information theory asks what correlations and communication survive such channels. Spectral entropy is $S(\rho) = -\sum_j \lambda_j \log_2 \lambda_j$, with eigenvalues λ_j and $0\log_2 0 = 0$. It is zero for a pure state and one bit for $I/2$.

Check. A Bell pair has joint entropy zero and local entropy one. Explain why adding the local entropies incorrectly treats the subsystems as independent.

Partial trace without guessing. For basis operators,

$$\operatorname{Tr}_B(|ab\rangle\langle cd|) = |a\rangle\langle c|\,\langle d \mid b\rangle.$$

Expand the Bell projector into its four terms: $\frac{1}{2}(|00\rangle\langle 00| + |00\rangle\langle 11| + |11\rangle\langle 00| + |11\rangle\langle 11|)$. The middle terms vanish under Tr_B, while the others give $\rho_A = (|0\rangle\langle 0| + |1\rangle\langle 1|)/2$. In an array, this is the index contraction $(\rho_A)_{ac} = \sum_b (\rho_{AB})_{ab,cb}$. Pair matching indices on the discarded subsystem, not on the retained one.

Solved spectral extension. Consider a second mixed state, distinct from the fair mixture $I/2$. Take

$$\rho = \frac{1}{4}\begin{pmatrix} 2 & \mathrm{i} \\ -\mathrm{i} & 2 \end{pmatrix}, \qquad \det(4\rho - \lambda I) = (2-\lambda)^2 - \mathrm{i}(-\mathrm{i}) = (2-\lambda)^2 - 1.$$

The roots are $1, 3$, so ρ has eigenvalues $1/4, 3/4$. For root 1, $a + \mathrm{i}b = 0$ gives $v_1 = (1, \mathrm{i})^T/\sqrt{2}$; for root 3, $-a + \mathrm{i}b = 0$ gives $v_3 = (1, -\mathrm{i})^T/\sqrt{2}$. Their conjugated inner product is $(1 + (-\mathrm{i})(-\mathrm{i}))/2 = 0$. Reconstruct $\rho = \frac{1}{4}v_1v_1^\dagger + \frac{3}{4}v_3v_3^\dagger$ and compute purity $1/16 + 9/16 = 5/8$. Direct multiplication verifies $\rho v_1 = \frac{1}{4}v_1$ and $\rho v_3 = \frac{3}{4}v_3$: the subscripts label roots of 4ρ, not eigenvalues of ρ.

3 Quantum computation

3.1 Qubit geometry and elementary gates

Source trail: [1], *Lessons 1–3;* [2], *Lectures 3, 9–10. References on p. 40.*

After removing a global phase, every pure qubit can be written

$$|\psi\rangle = \cos(\theta/2)|0\rangle + e^{i\phi}\sin(\theta/2)|1\rangle, \tag{3.1}$$

where $0 \le \theta \le \pi$ and $0 \le \phi < 2\pi$. At the poles ϕ is irrelevant. The Bloch vector is $r = (\sin\theta\cos\phi, \sin\theta\sin\phi, \cos\theta)$, with density matrix $\rho = (I + r_x X + r_y Y + r_z Z)/2$. Pure states lie on its unit sphere; mixed qubits occupy the interior. Coordinates are expectation values $r_j = \mathrm{Tr}(\rho\sigma_j)$, where σ_j is the corresponding Pauli matrix.

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}, \quad Y = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}, \quad Z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}. \tag{3.2}$$

Each is Hermitian and unitary and squares to I. They exchange or change the phases of basis amplitudes. Their anticommutation, for example $XZ = -ZX$, is a concrete sign that gate order matters.

The Hadamard gate and phase gates are

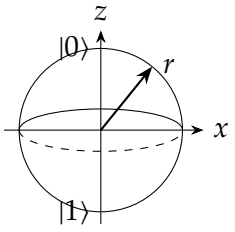
$$\mathsf{H} = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}, \quad S = \mathrm{diag}(1, i), \quad T = \mathrm{diag}(1, e^{i\pi/4}). \tag{3.3}$$

H switches between the Z and X eigenbases. S and T leave Z probabilities unchanged but change relative phase.

Worked example: Hadamard on zero, without verbal shortcuts

$$\mathsf{H}|0\rangle = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}\begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 \\ 1 \end{pmatrix} = |+\rangle.$$

Both probabilities are $1/2$. A second Hadamard returns $|0\rangle$ because the new amplitude for one is $(1-1)/2 = 0$. It reverses a coherent preparation; it would not reverse a random classical choice of zero or one.



Check. The state $(|0\rangle + i|1\rangle)/\sqrt{2}$ has $r = (0, 1, 0)$. Verify by direct Pauli expectations. The diagram is a projection of the sphere; it does not depict physical position.

3.2 Rotations and a complete gate sequence

A Pauli rotation is a continuous gate

$$R_P(\vartheta) = e^{-\mathrm{i}\vartheta P/2} = \cos(\vartheta/2)I - \mathrm{i}\sin(\vartheta/2)P, \tag{3.4}$$

where $P \in \{X, Y, Z\}$ and ϑ is a real angle. Expand the exponential as a series and use $P^{2k} = I$, $P^{2k+1} = P$ to obtain the last expression. The factor of two connects state-vector phases to the angle of the Bloch vector's rotation. In particular,

$$R_y(\vartheta) = \begin{pmatrix} \cos(\vartheta/2) & -\sin(\vartheta/2) \\ \sin(\vartheta/2) & \cos(\vartheta/2) \end{pmatrix}, \quad R_z(\vartheta) = \mathrm{diag}(e^{-\mathrm{i}\vartheta/2}, e^{\mathrm{i}\vartheta/2}). \tag{3.5}$$

$S = e^{\mathrm{i}\pi/4} R_z(\pi/2)$, so the two differ by a global phase when used as standalone gates. A global phase of a gate can become a relative phase if the gate is controlled; do not discard it in that setting without checking.

Worked example: Prepare, phase, interfere, measure

Starting from $|0\rangle$, apply H, then S, then H:

$$|0\rangle \longmapsto (|0\rangle + |1\rangle)/\sqrt{2} \tag{3.6}$$

$$\longmapsto (|0\rangle + \mathrm{i}|1\rangle)/\sqrt{2} \tag{3.7}$$

$$\longmapsto [(1+\mathrm{i})|0\rangle + (1-\mathrm{i})|1\rangle]/2. \tag{3.8}$$

Each final probability is $|1 \pm \mathrm{i}|^2/4 = 1/2$. Replacing S by Z gives $\mathsf{HZH}|0\rangle = |1\rangle$. Thus the phase angle determines the final distribution. Multiplying matrices first gives the same answer, but tracking intermediate states makes the mechanism visible.

More generally a phase gate $P(\varphi) = \mathrm{diag}(1, e^{\mathrm{i}\varphi})$ in this sequence yields amplitudes $(1 + e^{\mathrm{i}\varphi})/2$ and $(1 - e^{\mathrm{i}\varphi})/2$. Squaring them gives $\cos^2(\varphi/2)$ and $\sin^2(\varphi/2)$. This is a minimal interferometer and the algebraic ancestor of phase kickback algorithms.

A state-vector simulator applies a one-qubit gate to pairs of amplitudes whose basis strings differ at that bit. For each pair (a, b), replace it by $(U_{00}a + U_{01}b, U_{10}a + U_{11}b)$. Store the old pair before overwriting it. One such gate takes $O(2^n)$ scalar work in this direct simulation, whereas the physical circuit has one gate. This distinguishes simulation cost from circuit size.

Check. Derive $R_y(\pi/3)|0\rangle = (\sqrt{3}|0\rangle + |1\rangle)/2$. Its probability of one is $1/4$.

An implementation-sized calculation. On the general qubit $(a, b)^T$,

$$\mathsf{H}(a, b)^T = ((a+b)/\sqrt{2}, (a-b)/\sqrt{2})^T.$$

For the middle state above use $a = 1/\sqrt{2}$, $b = \mathrm{i}/\sqrt{2}$: the first output is $(1+\mathrm{i})/2$ and the second $(1-\mathrm{i})/2$. Compute their squared moduli using conjugates to obtain $1/2 + 1/2 = 1$ overall. Store old a, b before updating either entry: overwriting a first and reusing it for the second result implements the wrong map. This same safeguard applies to every two-amplitude update in a many-qubit simulation.

Exercise. Replace S with T. Use $|1 + e^{\mathrm{i}\pi/4}|^2 = 2 + 2\cos(\pi/4)$ to obtain $p(0) = (2 + \sqrt{2})/4$.

3.3 Controlled gates, Bell states and conditional outcomes

A controlled-U acts on a target only when its control is one:

$$C(U) = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes U. \tag{3.9}$$

The orthogonal control projectors ensure unitarity when U is unitary. CNOT is $C(X)$ and sends $|a, b\rangle$ to $|a, b \oplus a\rangle$, with $\oplus$ addition modulo two. In our basis order,

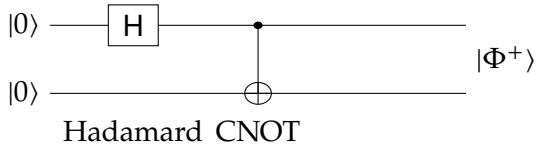
$$\mathrm{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}. \tag{3.10}$$

Worked example: Constructing and measuring a Bell pair

Prepare $|00\rangle$, apply $\mathsf{H} \otimes I$, then CNOT with the top qubit controlling the lower one:

$$|00\rangle \longmapsto (|00\rangle + |10\rangle)/\sqrt{2} \longmapsto |\Phi^+\rangle = (|00\rangle + |11\rangle)/\sqrt{2}.$$

The joint probabilities are $p(00) = p(11) = 1/2$ and $p(01) = p(10) = 0$. The first bit is uniform, but conditional on it being zero the second is certainly zero. These correlations do not permit communication: local statistics remain uniform regardless of which basis the distant observer chooses, when their outcome is unavailable.



The circuit above is original and uses a filled control dot and a circled target plus. Time runs left to right. Unlike a Boolean branching instruction, a controlled gate acts linearly on all amplitudes, without measuring the control.

For a general $\alpha|0\rangle + \beta|1\rangle$ with a zero target, CNOT produces $\alpha|00\rangle + \beta|11\rangle$. It is entangled when both coefficients are nonzero. It does not produce $(\alpha|0\rangle+\beta|1\rangle)^{\otimes 2}$: expanding that expression would generate four terms with squared coefficients and cross terms. CNOT copies computational-basis labels, not unknown states.

Check. Apply $I \otimes X$ to $|\Phi^+\rangle$. You get $(|01\rangle + |10\rangle)/\sqrt{2}$. Applying $Z \otimes I$ instead changes the relative plus to a minus.

Follow the Bell calculation as arrays. Multiplication by $\mathsf{H} \otimes I$ sends $(1,0,0,0)^T$ to $(1,0,1,0)^T/\sqrt{2}$. CNOT swaps coordinates 10 and 11, producing $(1,0,0,1)^T/\sqrt{2}$. Hence $\Pr(q_0 = 0) = |a_{00}|^2 + |a_{01}|^2 = 1/2$ and $\Pr(q_1 = 0 \mid q_0 = 0) = p(00)/p(q_0 = 0) = 1$. In code, computing a marginal requires summing compatible *probabilities*; adding $a_{00} + a_{01}$ would instead describe a different interference experiment.

3.4 Circuit reasoning, no-cloning and error correction

Source trail: [1], *Lessons 3, 13–14;* [3], *Chapter 7. References on p. 40.*

A circuit specifies a sequence of local unitaries, preparations and measurements. Its width is the number of live qubits; size counts gates; depth counts layers of gates that can run in parallel on disjoint qubits. Connectivity restrictions can add swap gates, and synthesising precise rotations from a finite native gate set adds cost. A compact abstract circuit is therefore only the beginning of a resource estimate.

No-cloning theorem. Suppose a unitary copied any state into a blank register: $U|\psi\rangle|0\rangle = |\psi\rangle|\psi\rangle$ and similarly for $|\phi\rangle$. Inner-product preservation would require

$$\langle\phi \mid \psi\rangle = (\langle\phi \mid \psi\rangle)^2. \tag{3.11}$$

A pair with real overlap $1/\sqrt{2}$, such as $|0\rangle$ and $|+\rangle$, contradicts this equation. Hence no universal perfect copier exists. Orthogonal labels can be copied; arbitrary unknown states cannot. More general deterministic physical copying is also impossible, since a unitary dilation would impose the corresponding overlap constraint.

Worked example: Why measuring first cannot rescue copying

Measuring $|+\rangle$ in the Z basis gives a random bit. Preparing two copies of that bit produces the mixture $\frac{1}{2}|00\rangle\langle 00| + \frac{1}{2}|11\rangle\langle 11|$. Two true copies of $|+\rangle$ would give $X \otimes X$ expectation one; the mixture gives zero. A recipe that reproduces one measurement distribution need not reproduce the state.

Error correction avoids cloning by encoding information into correlations. The three-qubit repetition encoding is

$$\alpha|0\rangle + \beta|1\rangle \longmapsto \alpha|000\rangle + \beta|111\rangle. \tag{3.12}$$

Measure parity observables Z_1Z_2 and Z_2Z_3, each with outcomes ± 1. Both codewords have syndrome $(+,+)$, so these checks reveal no logical amplitudes. A single X_1, X_2 or X_3 error gives $(-,+)$, $(-,-)$ or $(+,-)$, respectively. Applying the identified X_j restores the encoded state.

This code corrects one bit flip, not arbitrary noise. For example, Z_1 changes the logical relative phase without changing either syndrome. General quantum codes must protect both phase and bit information; fault-tolerant circuits must also prevent error propagation during correction. Threshold statements assume a specified noise model and suitable scalable operations. They are not a promise that any noisy circuit can be repaired for free.

Check. Derive the syndrome after X_2 using $ZX = -XZ$. Then explain why directly measuring all three data qubits would destroy an unknown logical superposition.

Worked syndrome. After X_2, the encoded state is $\alpha|010\rangle + \beta|101\rangle$. On 010, Z_1Z_2 contributes $(+1)(-1) = -1$ and Z_2Z_3 contributes $(-1)(+1) = -1$. On 101 both products are again -1. Therefore the whole superposition has syndrome $(-,-)$, regardless of α, β. Applying X_2 returns the original codeword. By contrast, measuring all data bits separately distinguishes 010 from 101 and removes their coherence. The protected information lives in the relative amplitudes, not in either classical word alone.

4 Quantum algorithms

4.1 Oracles, reversible computation and phase kickback

Source trail: [1], *Lessons 5–8;* [2], *Lectures 16, 19–25. References on p. 40.*

An algorithm must arrange interference so a useful property survives measurement. Preparing 2^n amplitudes does not make all 2^n function values independently readable. An oracle is a precisely defined access model; its construction may be the dominant cost in a real application.

For a Boolean function $f : \{0,1\}^n \to \{0,1\}$, the standard reversible oracle is

$$U_f|x\rangle|y\rangle = |x\rangle|y \oplus f(x)\rangle. \tag{4.1}$$

It is unitary because it permutes basis states and is its own inverse. Merely mapping $|x\rangle$ to $|f(x)\rangle$ would not generally be reversible: many inputs can share one output. A classical program for f can be made reversible using work bits. After computing and copying the result into the target, reverse the computation to reset the work bits. This is called uncomputation; uncleared work can retain which-input information and spoil interference.

Prepare $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$. Because $X|-\rangle = -|-\rangle$,

$$U_f|x\rangle|-\rangle = (-1)^{f(x)}|x\rangle|-\rangle. \tag{4.2}$$

The target returns unchanged, while the function value becomes a relative sign on the input amplitudes. This phase kickback implements the phase oracle $O_f|x\rangle = (-1)^{f(x)}|x\rangle$ using one standard oracle call.

Worked example: Building an oracle for a parity predicate

For two bits, let $f(x_0, x_1) = x_0 \oplus x_1$. CNOT from each input onto the target implements U_f. With a minus target, input amplitudes for 01 and 10 acquire a minus; 00 and 11 do not. The input vector $\frac{1}{2}(1,1,1,1)^T$ becomes $\frac{1}{2}(1,-1,-1,1)^T$.

A query complexity counts uses of U_f or O_f. A gate complexity counts elementary operations including construction of the oracle. Runtime also includes preparation, repetitions, routing, error correction and classical processing. If a predicate requires scanning a large database, saying "one query" does not make that scan free.

Algorithm design habit. Specify the input promise, allowed access, exact output and success probability before claiming a speedup. Compare algorithms under the same access model. A black-box lower bound need not apply to a structured function with a short analytic description.

Check. Repeat the kickback calculation using $|+\rangle$ as target. Since $X|+\rangle = |+\rangle$, no input-dependent phase appears. The choice of target eigenstate is essential.

Derive kickback in the two possible branches. If $f(x) = 0$, the target is unchanged. If $f(x) = 1$,

$$U_f|x\rangle\frac{|0\rangle - |1\rangle}{\sqrt{2}} = |x\rangle\frac{|1\rangle - |0\rangle}{\sqrt{2}} = -|x\rangle|-\rangle.$$

Linearity applies this rule to every term of a superposition. A classical simulator can multiply marked amplitudes by -1 directly; a hardware circuit must realise the predicate reversibly. These two implementations have different costs even though they implement the same ideal oracle.

4.2 Deutsch–Jozsa and Bernstein–Vazirani

Deutsch–Jozsa assumes a promise: f is either constant or balanced, meaning exactly half of its $N = 2^n$ inputs give one. Begin with $|0\rangle^{\otimes n}|1\rangle$, apply Hadamards to all qubits, query U_f, and apply Hadamards to the input register again. The target stays separable in $|-\rangle$.

For binary strings x, z, define $x \cdot z = \bigoplus_j x_j z_j$. The tensor Hadamard identity is

$$\mathsf{H}^{\otimes n}|x\rangle = \frac{1}{\sqrt{N}} \sum_z (-1)^{x\cdot z}|z\rangle. \tag{4.3}$$

It follows by multiplying the one-bit expansions. Thus the final input amplitude is

$$a_z = \frac{1}{N} \sum_x (-1)^{f(x)+x\cdot z}. \tag{4.4}$$

At $z = 0^n$, a constant function has amplitude $+1$ or -1, while a balanced function has equal positive and negative contributions and amplitude zero. Measuring all zero therefore decides the promised cases exactly with one query. An exact deterministic classical algorithm can need $N/2 + 1$ queries; a bounded-error randomised classical test needs only a constant number for fixed error. The comparison is not an exponential separation against all classical algorithms.

Worked example: A two-bit interference calculation

For $f(x) = x_0 \oplus x_1$, the phase vector is $(1, -1, -1, 1)/2$. In Eq. (4.4), $z = 11$ cancels the parity sign for every x, so $a_{11} = 1$. For each other z, the four terms sum to zero. The output is 11 with certainty, not just "some nonzero string."

That stronger observation gives Bernstein–Vazirani. Suppose $f_s(x) = s \cdot x \oplus b$ for an unknown string s and bit b. Substitution gives

$$a_z = \frac{(-1)^b}{N} \sum_x (-1)^{(s\oplus z)\cdot x} = (-1)^b \, \mathbf{1}[z = s]. \tag{4.5}$$

Here $\mathbf{1}[\cdot]$ is one if its condition holds and zero otherwise. If $s \neq z$, pair strings differing in a bit where $s \oplus z$ is one: their terms cancel. The measurement reveals all of s in one query. The offset b becomes only a global sign and is not learned by this circuit. A classical exact strategy can learn $b = f(0)$ and then obtain s from the n unit-vector queries.

Check. With $s = 101$ and $b = 1$, compute the output state. It is $-|101\rangle$, which has the same observable distribution as $|101\rangle$.

Show the cancellations explicitly. In the parity example, the four output sums are

$$(a_{00}, a_{01}, a_{10}, a_{11}) = \tfrac{1}{4}(1 - 1 - 1 + 1,\ 1 + 1 - 1 - 1,\ 1 - 1 + 1 - 1,\ 1 + 1 + 1 + 1).$$

Thus only a_{11} survives. A direct implementation can loop over input x and output z, accumulating $(-1)^{f(x)+x\cdot z}/N$. Use this slow but transparent calculation to test a gate-based simulator on small n. It checks the dot-product parity and bit order independently of the circuit implementation.

4.3 Grover search as a rotation

Let $f(x)$ mark M good inputs among N, with $0 < M < N$. Define normalised uniform vectors $|G\rangle$ over good inputs and $|B\rangle$ over the rest. The uniform initial state is

$$|s\rangle = \sqrt{M/N}|G\rangle + \sqrt{1 - M/N}|B\rangle = \sin\theta|G\rangle + \cos\theta|B\rangle, \tag{4.6}$$

where $\theta = \arcsin\sqrt{M/N}$. The phase oracle reflects the good component: $O_f = I - 2\Pi_G$, where Π_G projects onto marked basis states. The diffusion operation $D = 2|s\rangle\langle s| - I$ reflects about $|s\rangle$. Its circuit is $\mathsf{H}^{\otimes n}(2|0\rangle\langle 0| - I)\mathsf{H}^{\otimes n}$.

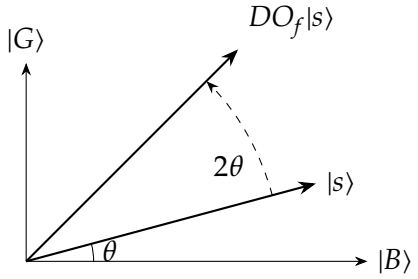
In the ordered basis $(|G\rangle, |B\rangle)$, multiplying these two reflections gives

$$DO_f = \begin{pmatrix} \cos 2\theta & \sin 2\theta \\ -\sin 2\theta & \cos 2\theta \end{pmatrix}. \tag{4.7}$$

Applied to $(\sin\theta, \cos\theta)^T$, it yields $(\sin 3\theta, \cos 3\theta)^T$. By induction, after r iterations,

$$p_r = \sin^2((2r+1)\theta). \tag{4.8}$$

Choose the nonnegative integer r nearest $\pi/(4\theta) - 1/2$ when θ is known. For M/N small, $\theta \approx \sqrt{M/N}$, so $r \approx (\pi/4)\sqrt{N/M}$. This is the $O(\sqrt{N/M})$ query scaling. Overshooting rotates the state away from the good subspace, so indefinite iteration does not keep improving success.



The illustration uses $\theta = 15°$ for clarity; the algebra holds for every allowed M/N. Grover's search is optimal in order for unstructured oracle search with bounded error [1,2,8]. It does not turn general NP-hard optimisation into polynomial time, since N can itself be exponential in the input length.

Check. For $N = 4, M = 1$, $\theta = \pi/6$ and one iteration succeeds exactly.

Derive the rotation rather than memorising it. Write $s_\theta = \sin\theta$, $c_\theta = \cos\theta$. Restricted to the two-dimensional invariant space,

$$D = \begin{pmatrix} 2s_\theta^2 - 1 & 2s_\theta c_\theta \\ 2s_\theta c_\theta & 2c_\theta^2 - 1 \end{pmatrix}, \qquad O_f = \begin{pmatrix} -1 & 0 \\ 0 & 1 \end{pmatrix}.$$

Multiplication negates the first column of D. Use $1 - 2\sin^2\theta = \cos 2\theta$ and $2\sin\theta\cos\theta = \sin 2\theta$ to obtain the stated matrix. Its good component after one step is $\cos 2\theta \sin\theta + \sin 2\theta \cos\theta = \sin 3\theta$. The negative lower-left entry follows from ordering coordinates as (G, B); it does not signify a negative probability.

4.4 Numerical search and amplitude amplification

Source trail: [1], *Lesson 8;* [8], [9]. *References on p. 40.*

Diffusion has a useful coordinate interpretation. For a real amplitude vector with mean $\bar{a} = N^{-1}\sum_x a_x$, it maps a_x to $2\bar{a} - a_x$. This follows from $2|s\rangle\langle s| - I$; the same formula also holds for complex amplitudes.

Worked example: One marked element in a four-state space

Start with $(1,1,1,1)/2$ and mark 11. The oracle gives $(1,1,1,-1)/2$, whose mean is $1/4$. Diffusion maps each unmarked amplitude to $2(1/4) - 1/2 = 0$ and the marked amplitude to $2(1/4) + 1/2 = 1$. The state is exactly $|11\rangle$. Squared amplitudes still sum to one.

For $N = 8, M = 1$, $\theta \approx 0.3614$. Two iterations give $p_2 = \sin^2(5\theta) = 121/128 \approx 0.9453$; three give a lower probability. In a real application, validate the measured candidate using the predicate, because the algorithm can fail.

Amplitude amplification replaces the uniform preparation by a unitary algorithm A with

$$A|0\rangle = \sqrt{p}|G\rangle + \sqrt{1-p}|B\rangle. \tag{4.9}$$

Here $|G\rangle$ and $|B\rangle$ are the normalised projections of this particular prepared state onto good and bad subspaces. They need not be uniform. Reflect about $A|0\rangle$ using $A(2|0\rangle\langle 0| - I)A^\dagger$ and about the good subspace using $I - 2\Pi_G$. Their product has the same two-dimensional rotation analysis, with $\sin^2\theta = p$.

For small known p, $O(1/\sqrt{p})$ iterations replace the $O(1/p)$ independent classical repetitions needed for constant success. Count uses of A, $A^\dagger$ and the good-state test, not just one chosen subroutine. If p is unknown, a schedule of randomised iteration counts can avoid systematic overshooting; the naive fixed-iteration rule requires knowing or estimating p [9].

For Athena, a speculative quantum use would amplify feasible workflow plans from a reversible plan generator. The difficult prerequisite is an efficient coherent generator and feasibility oracle. Ordinary Python workflows with network calls, irreversible side effects and intermediate measurements cannot simply serve as A. A classical scheduler using structure may beat searching an exponentially large plan space even with a quadratic oracle saving.

Check. If a preparation succeeds with $p = 10^{-4}$, compare the orders $1/p = 10^4$ and $1/\sqrt{p} = 100$. Then list at least three costs absent from this query comparison.

A solved $N = 8$ amplitude calculation. Put $a = 1/\sqrt{8}$ and mark one item. After the first oracle the mean is $(7a - a)/8 = 3a/4$. Diffusion gives each bad amplitude $a/2$ and the good amplitude $5a/2$. The next oracle flips only the latter, giving mean $(7a/2 - 5a/2)/8 = a/8$. Diffusing again gives bad amplitudes $-a/4$ and good amplitude $11a/4$. Thus $p_2 = (11a/4)^2 = 121/128$; total probability is $(121 + 7)/128 = 1$. Negative bad amplitudes are essential for this normalised state.

Implementation exercise. Starting with an array of N equal amplitudes, implement "flip marked signs; compute mean; replace each entry by twice the mean minus itself." Save the mean before modifying entries, and verify the squared norm after each iteration.

4.5 The quantum Fourier transform

Source trail: [1], *Lessons 6–7;* [2], *Lecture 19. References on p. 40.*

For N basis labels, the positive-sign Fourier convention is

$$F_N|j\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} \omega^{jk}|k\rangle, \qquad \omega = e^{2\pi \mathrm{i}/N}. \tag{4.10}$$

Extend linearly to all states. The inverse has the negative sign. The inner product of column j with column ℓ is

$$(F_N^\dagger F_N)_{j\ell} = \frac{1}{N} \sum_{k=0}^{N-1} (\omega^{jk})^* \omega^{\ell k} = \frac{1}{N} \sum_{k=0}^{N-1} \omega^{k(\ell-j)}.$$

If $j = \ell$ it is one. Otherwise set $q = \omega^{\ell-j} \neq 1$ and use $\sum_{k=0}^{N-1} q^k = (1-q^N)/(1-q) = 0$. This roots-of-unity cancellation is the transform's central interference mechanism.

Worked example: A complete two-qubit transform

For $N = 4$, $\omega = \mathrm{i}$ and

$$F_4 = \frac{1}{2} \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & \mathrm{i} & -1 & -\mathrm{i} \\ 1 & -1 & 1 & -1 \\ 1 & -\mathrm{i} & -1 & \mathrm{i} \end{pmatrix}.$$

For $j = 1$, substitute $k = 0, 1, 2, 3$: $\mathrm{i}^{jk} = 1, \mathrm{i}, -1, -\mathrm{i}$. Thus $F_4|1\rangle = (|0\rangle + \mathrm{i}|1\rangle - |2\rangle - \mathrm{i}|3\rangle)/2$. Its measurement is uniform: phase information alone does not identify j. For input $(|0\rangle + |2\rangle)/\sqrt{2}$, output amplitude k is

$$b_k = \frac{1 + \mathrm{i}^{2k}}{2\sqrt{2}} = \frac{1 + (-1)^k}{2\sqrt{2}}.$$

It is $1/\sqrt{2}$ at $k = 0, 2$ and zero at $k = 1, 3$, giving $(|0\rangle + |2\rangle)/\sqrt{2}$.

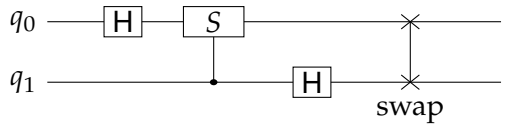
For $N = 2^n$, write input bits $j_1 \cdots j_n$ most significant first. Write $k = \sum_{r=1}^{n} k_r 2^{n-r}$, with bits $k_r \in \{0, 1\}$. Then

$$e^{2\pi \mathrm{i} jk/2^n} = \prod_{r=1}^{n} e^{2\pi \mathrm{i} j k_r/2^r}.$$

Summing independently over each $k_r = 0, 1$ distributes into a product of two-term sums, so

$$F_{2^n}|j\rangle = \bigotimes_{r=1}^{n} \frac{|0\rangle + e^{2\pi \mathrm{i} j/2^r}|1\rangle}{\sqrt{2}}. \tag{4.11}$$

The leftmost output factor here is most significant. The fractional part of $j/2^r$ is the binary fraction $0.j_{n-r+1} \cdots j_n$. Hadamards create equal magnitudes; controlled phase gates supply these fractions, and final swaps reverse the wire order in the usual circuit construction.



This is F_4 under our bit convention. Exact QFT uses n Hadamards, $n(n-1)/2$ controlled phases and $\lfloor n/2 \rfloor$ swaps: $O(n^2)$ abstract gates. This does not output the full classical Fourier array; extracting all 2^n amplitudes removes that apparent shortcut.

Paper-to-circuit check. For input $|01\rangle$, the drawn circuit gives $|01\rangle \to (|01\rangle + |11\rangle)/\sqrt{2} \to (|01\rangle + \mathrm{i}|11\rangle)/\sqrt{2}$. The lower Hadamard then gives $(|00\rangle - |01\rangle + \mathrm{i}|10\rangle - \mathrm{i}|11\rangle)/2$; swapping wires yields $(|00\rangle + \mathrm{i}|01\rangle - |10\rangle - \mathrm{i}|11\rangle)/2$, exactly the matrix result. To implement the inverse, reverse gate order and conjugate every phase, rather than merely changing the final swap.

4.6 Quantum phase estimation

Suppose $U|u\rangle = e^{2\pi i\varphi}|u\rangle$, where $0 \le \varphi < 1$. Phase estimation converts this eigenphase into classical bits. Prepare an m-qubit counting register uniformly and use controlled powers of U to produce

$$\frac{1}{\sqrt{2^m}} \sum_{j=0}^{2^m-1} e^{2\pi ij\varphi}|j\rangle|u\rangle. \tag{4.12}$$

The factor arises from phase kickback. Counting bit of weight 2^k controls U^{2^k}. Apply $F_{2^m}^\dagger$ to the counting register. With $L = 2^m$, the amplitude for integer y is

$$a_y = \frac{1}{L} \sum_{j=0}^{L-1} e^{2\pi ij(\varphi - y/L)}. \tag{4.13}$$

If $\varphi = s/L$ for an integer s, roots-of-unity cancellation gives $y = s$ with certainty. Otherwise put $\delta = \varphi - y/L$. The geometric series gives

$$|a_y|^2 = \frac{\sin^2(\pi L\delta)}{L^2 \sin^2(\pi\delta)}, \tag{4.14}$$

with the continuous limit used when the denominator vanishes. The distribution concentrates near $y/L = \varphi$ modulo one; it is not a noiseless rounding oracle.

Worked example: Three-bit exact phase

Let $U = \mathrm{diag}(1, e^{2\pi i(3/8)})$ and $|u\rangle = |1\rangle$. Controlled U, U^2, U^4 encode the phase $3/8$. The inverse eight-point QFT returns $y = 3$, so the measured bit string is 011. The binary fraction $0.011_2 = 1/4 + 1/8 = 3/8$ recovers the phase.

If the input is $\sum_\ell c_\ell|u_\ell\rangle$, linearity entangles each eigenvector with its phase distribution. An exact, nondegenerate phase measurement selects an eigenvector with probability $|c_\ell|^2$; equal eigenphases retain coherence within their eigenspace.

Precision has a cost. Resolving phase to scale ϵ uses roughly $m = \log_2(1/\epsilon)$ counting bits, but the largest controlled power is about $1/\epsilon$. If only a black-box U is available, implementing all powers by repetition takes $L - 1$ calls, not m. Special structure may implement powers more efficiently. Extra counting bits and repetition raise confidence; hardware noise and imperfect eigenstate preparation can still dominate.

Connection. For $U = e^{-iHt/\hbar}$, the phase is $-Et/(2\pi\hbar)$ modulo one for energy E. An energy range and suitable time must be specified to resolve the modulo ambiguity.

Resolve the exact example by its sum. With $L = 8$ and $\varphi = 3/8$, the $y = 3$ amplitude is $\frac{1}{8}\sum_{j=0}^{7} 1 = 1$. At $y = 2$ it is $\frac{1}{8}\sum_{j=0}^{7} e^{2\pi ij/8} = 0$. For any other $y \neq 3$ in $\{0, \dots, 7\}$, the same root-sum argument gives zero. At a nonbinary phase such as $\varphi = 1/3$, compute the eight $|a_y|^2$ using the geometric formula and check that they sum to one. A numerical test should compare both the direct complex sum and the sine-ratio form; use the limit at zero denominators.

4.7 Shor: factoring by finding an order

Source trail: [1], *Lesson 7;* [2], *Lectures 21–22. References on p. 40.*

Let N be an odd composite integer. Choose $1 < a < N$ and compute $g = \gcd(a, N)$ classically. If $g > 1$ it is already a factor. Otherwise multiplication by a modulo N is reversible. The order r is the least positive integer with $a^r \equiv 1 \pmod{N}$.

If r is even, then $(a^{r/2} - 1)(a^{r/2} + 1) \equiv 0 \pmod{N}$. If also $a^{r/2} \not\equiv -1 \pmod{N}$, the gcds $\gcd(a^{r/2} \pm 1, N)$ yield nontrivial factors. Because r is minimal, $a^{r/2} \not\equiv 1 \pmod{N}$. Unhelpful choices of a require another trial; standard factoring also handles even integers and prime powers classically.

The quantum subroutine estimates eigenphases of $U_a|y\rangle = |ay \bmod N\rangle$. Define its action as identity on unused labels $y \geq N$ to obtain a full-register unitary. On the cycle $1, a, \ldots, a^{r-1}$, eigenvectors

$$|u_s\rangle = \frac{1}{\sqrt{r}} \sum_{k=0}^{r-1} e^{-2\pi i s k/r} |a^k \bmod N\rangle \tag{4.15}$$

have eigenvalues $e^{2\pi i s/r}$, for $0 \leq s < r$. Shifting k by one verifies this equation. The easily prepared $|1\rangle$ is $r^{-1/2} \sum_s |u_s\rangle$, so phase estimation samples approximations to s/r.

Choose $L = 2^m \geq N^2$. A sufficiently close estimate y/L can reveal a reduced fraction s/r by continued fractions: when $|y/L - s/r| < 1/(2r^2)$ and $\gcd(s, r) = 1$, that fraction appears among its convergents. Continued fractions repeatedly take an integer part and invert the remainder. Check candidate denominators by modular exponentiation; reduced denominators may be proper divisors of the true order, so repeated samples or additional processing can be necessary.

Worked example: Factoring 15

Choose $a = 2$. The cycle is $1, 2, 4, 8, 1$, so $r = 4$. A phase sample $1/4$ identifies denominator four. Then $2^{r/2} = 4$, and $\gcd(4 - 1, 15) = 3$, $\gcd(4 + 1, 15) = 5$. A zero phase sample gives no order information; denominator two alone does not verify since $2^2 \not\equiv 1 \pmod{15}$.

For input length $n = \lceil \log_2 N \rceil$, standard reversible arithmetic gives polynomial gate complexity, often quoted as $O(n^3)$ in elementary constructions; arithmetic choices change this bound. QFT is not the sole cost: controlled modular multiplication dominates. No polynomial-time classical factoring algorithm is known, but an unconditional exponential lower bound is also unknown. Practical cryptographic factoring would require a sufficiently large fault-tolerant computer, including substantial correction overhead; tiny compiled demonstrations do not establish such capability.

Unpack the eigenvector shift. Put $\ell = k + 1$ modulo r in $U_a|u_s\rangle$. Then $e^{-2\pi i s k/r} = e^{2\pi i s/r} e^{-2\pi i s \ell/r}$, so the common factor is the claimed eigenvalue. For the sample $1/4$, continued fractions give $[0; 4]$ since taking the reciprocal of $1/4$ gives four. Verify the candidate directly: $2^4 \bmod 15 = 1$, but $2^1 \bmod 15 = 2$ and $2^2 \bmod 15 = 4$. The quantum subroutine suggests arithmetic data; the classical verification prevents accepting an unhelpful denominator as a factorisation.

5 Quantum optimisation

5.1 Binary models and penalty reasoning

Source trail: [4], *QUBOs and Ising Models; Reformulating a Problem. References on p. 40.*

Combinatorial optimisation chooses a discrete configuration that minimises a cost. A task assignment, route or execution plan may have exponentially many candidates, but structure often permits efficient special cases. Formulating a problem as QUBO does not by itself make it easier.

For binary $x_i \in \{0, 1\}$, a quadratic unconstrained binary objective is

$$E(x) = C + \sum_i a_i x_i + \sum_{i<j} b_{ij} x_i x_j = C + x^T Q x. \tag{5.1}$$

Here C is a constant offset and Q is upper triangular: $Q_{ii} = a_i$ and $Q_{ij} = b_{ij}$ for $i < j$. In a symmetric convention, off-diagonal entries must instead be $b_{ij}/2$, because $x^T Q x$ counts them twice. Binary idempotence $x_i^2 = x_i$ reduces squared single-variable terms to linear terms.

"Unconstrained" means constraints are encoded in the energy. To require exactly one of k bits, add

$$A\left(\sum_{j=1}^{k} x_j - 1\right)^2 = A\left(1 - \sum_j x_j + 2\sum_{j<\ell} x_j x_\ell\right), \tag{5.2}$$

with penalty weight $A > 0$. The square is zero for feasible configurations and at least one otherwise. An at-most-one condition needs only $A \sum_{j<\ell} x_j x_\ell$, since zero selections are allowed.

Worked example: Why penalty size needs a bound

Minimise $-5x_1 - 4x_2$ subject to $x_1 + x_2 = 1$. Feasible costs are $-5, -4$. Selecting both gives $-9 + A$. If $A = 2$, this infeasible choice wins. If $A > 4$, it cannot beat -5. A penalty cannot be chosen independently of the original objective's scale.

A safe general argument is to bound the full objective range by R. If every violation costs at least A and $A > R$, no infeasible assignment can improve over all feasible ones, provided a feasible assignment exists. This bound can be conservative. Excessively large penalties can also reduce effective coefficient precision after hardware scaling and create difficult search landscapes.

For an integer inequality $\sum_i w_i x_i \le B$, introduce nonnegative integer slack s and penalise $(\sum_i w_i x_i + s - B)^2$. Encode s using bits with weights that cover every required slack value. This adds variables and couplings. For real weights, choosing a finite precision is part of the model and must be documented.

Check. Expand $A(x_1 + x_2 + x_3 - 1)^2$. Each linear coefficient is $-A$, each pair coefficient is $2A$, and the constant is A.

Derive the general square. Let $S = \sum_j x_j$. Then $(S-1)^2 = S^2 - 2S + 1$ and $S^2 = \sum_j x_j^2 + 2\sum_{j<\ell} x_j x_\ell$. Substitute $x_j^2 = x_j$ to obtain the displayed one-hot penalty. The factor two comes from both ordered products $x_j x_\ell$ and $x_\ell x_j$.

Compiler rule. To add $A(\sum_i w_i x_i - b)^2$ to an upper-triangular QUBO, increment C by Ab^2, Q_{ii} by $A(w_i^2 - 2bw_i)$, and Q_{ij} by $2Aw_i w_j$ for $i < j$. Here b is the target sum. Derive these increments by expansion; apply them once per constraint and *add* overlapping contributions. Replacing an existing coefficient loses information from earlier constraints.

5.2 A complete task-allocation QUBO

Consider two independent tasks, 1 and 2, and two agents, A and B. Each task must be assigned once; each agent can execute at most one task in this batch. Assignment costs are

	Agent A	Agent B
Task 1	1	4
Task 2	2	1

Let $(x_1, x_2, x_3, x_4) = (x_{1A}, x_{1B}, x_{2A}, x_{2B})$. A one means the indicated assignment is chosen. The objective and constraints are

$$f = x_1 + 4x_2 + 2x_3 + x_4, \quad x_1 + x_2 = 1, \quad x_3 + x_4 = 1, \tag{5.3}$$

with forbidden double bookings $x_1x_3 = 0$ and $x_2x_4 = 0$. Using $A = 10$ for every constraint, define

$$E = f + 10[(x_1 + x_2 - 1)^2 + (x_3 + x_4 - 1)^2 + x_1x_3 + x_2x_4]. \tag{5.4}$$

Expand each square separately, using $x_i^2 = x_i$:

$$10(x_1 + x_2 - 1)^2 = 10 - 10x_1 - 10x_2 + 20x_1x_2.$$

The task-2 square similarly contributes $10 - 10x_3 - 10x_4 + 20x_3x_4$. Add original costs and capacity pairs to obtain

$$E = 20 - 9x_1 - 6x_2 - 8x_3 - 9x_4 + 20x_1x_2 + 10x_1x_3 + 10x_2x_4 + 20x_3x_4.$$

Read each linear coefficient onto the diagonal and each pair coefficient above it. This gives $C = 20$ and

$$Q = \begin{pmatrix} -9 & 20 & 10 & 0 \\ 0 & -6 & 0 & 10 \\ 0 & 0 & -8 & 20 \\ 0 & 0 & 0 & -9 \end{pmatrix}. \tag{5.5}$$

All original costs are nonnegative, and a feasible solution costs two. Every infeasible configuration incurs at least ten, so it cannot win. This direct argument is tighter and clearer than choosing a penalty by habit.

Worked example: Checking feasible and infeasible bit strings

The feasible vectors are $(1, 0, 0, 1)$ and $(0, 1, 1, 0)$ with costs 2 and 6. For the first, $x^TQx = -9 - 9 = -18$ and $E = 20 - 18 = 2$. Assigning both tasks to A gives $(1, 0, 1, 0)$ and $E = 20 - 9 - 8 + 10 = 13$. Selecting nothing gives $E = 20$; selecting only task 1 for A gives $E = 11$.

This example distinguishes business cost f from penalised energy E. Report f only for feasible solutions, and report violation counts separately. A low energy from the wrong convention or a missing constant can mislead validation. Constants do not change minimisers but are needed to compare numerical energies.

The toy problem is an assignment problem with an efficient classical solution; a QUBO heuristic is unnecessary for solving it. Its purpose is to expose encoding errors before adding realistic couplings. Athena's harder cases arise from resource limits, shared context, variable durations and dependencies, not from attaching the word quantum to an elementary assignment.

Check. Compute $E(1, 1, 0, 0) = 25$: task 1 is duplicated and task 2 is absent. Enumerate all 16 strings on paper or in a short program to confirm the unique minimum.

5.3 From QUBO to Ising and a problem Hamiltonian

Choose the convention $s_i = 2x_i - 1$, so $x_i = (1+s_i)/2$ and $s_i \in \{-1, +1\}$. Some libraries use the opposite convention; translate signs explicitly. Substituting into the scalar QUBO yields

$$E(s) = C' + \sum_i h_i s_i + \sum_{i<j} J_{ij} s_i s_j, \tag{5.6}$$

$$J_{ij} = b_{ij}/4, \qquad h_i = a_i/2 + \tfrac{1}{4} \sum_{j \neq i} b_{\min(i,j),\max(i,j)}, \tag{5.7}$$

$$C' = C + \tfrac{1}{2} \sum_i a_i + \tfrac{1}{4} \sum_{i<j} b_{ij}. \tag{5.8}$$

Every term follows from $x_i x_j = (1 + s_i + s_j + s_i s_j)/4$. This is an exact change of variables, not an approximation or a different optimisation problem.

Worked example: Transforming the four-bit allocation model

In Eq. (5.5), $a = (-9, -6, -8, -9)$ and the nonzero pairs are $b_{12} = 20$, $b_{13} = 10$, $b_{24} = 10$, $b_{34} = 20$. Thus

$$h = (3, 4.5, 3.5, 3), \quad (J_{12}, J_{13}, J_{24}, J_{34}) = (5, 2.5, 2.5, 5), \quad C' = 19.$$

The best binary vector $(1, 0, 0, 1)$ maps to $s = (1, -1, -1, 1)$. Its field sum is $3 - 4.5 - 3.5 + 3 = -2$. Its coupling sum is $-5 - 2.5 - 2.5 - 5 = -15$, so $E = 19 - 2 - 15 = 2$, agreeing with QUBO. To reproduce the coefficients, $h_1 = -9/2 + (20 + 10)/4 = 3$ and $C' = 20 - 32/2 + 60/4 = 19$; repeat for the other fields.

To define a gate-model cost Hamiltonian in the computational basis, remember $Z_i|x_i\rangle = (-1)^{x_i}|x_i\rangle$. Therefore the binary-number operator is $\hat{x}_i = (I - Z_i)/2$. Define

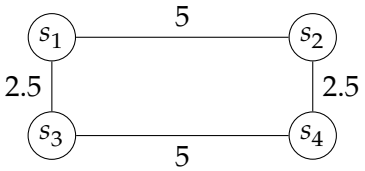
$$H_C = CI + \sum_i a_i \hat{x}_i + \sum_{i<j} b_{ij} \hat{x}_i \hat{x}_j. \tag{5.9}$$

It is diagonal and obeys $H_C|x\rangle = E(x)|x\rangle$. Its ground state, an eigenstate with smallest eigenvalue, encodes an optimal assignment. With our spin convention $s = 2x - 1$, the spin operator corresponds to $-Z$, so the h_i terms become $-h_i Z_i$ and pair terms keep their sign. Direct substitution avoids a common convention error.

A hardware annealer implements an Ising-type energy model using physical biases and couplings. Its reported spin labels need not use the same computational-basis interpretation as gate-model software. Check the backend convention and retain the offset in verification.

Check. For $E(x) = 2x_1 + 4x_1x_2$, obtain $C' = 2$, $h_1 = 2$, $h_2 = 1$, $J_{12} = 1$. Evaluate all four configurations in both forms.

Collect one pair term by hand. The contribution $20x_1x_2$ becomes $5 + 5s_1 + 5s_2 + 5s_1s_2$. It therefore changes four entries of the Ising model, not just J_{12}. The linear term $-9x_1$ contributes $-4.5 - 4.5s_1$. Adding the pair $10x_1x_3$ contributes another 2.5 to h_1, giving $-4.5 + 5 + 2.5 = 3$. This coefficient ledger is a useful independent check against a library conversion.



Only drawn pairs contribute.
Missing edges have $J_{ij} = 0$.

Read the coupling graph. The edge labels are J_{ij}, not penalties before conversion. Node fields remain $h = (3, 4.5, 3.5, 3)$. A positive edge favours opposite spins if considered alone; competing fields and constraints determine the full minimum.

5.4 Annealing, adiabatic evolution and classical alternatives

Source trail: [4], *What is Quantum Annealing?;* [10]. *References on p. 40.*

Simulated annealing (SA) is a classical stochastic algorithm. Starting from a binary string, propose a move, often one bit flip. If its energy change is ΔE, accept an improvement and accept a worsening move with probability

$$p_{\text{accept}} = \min(1, e^{-\Delta E/T}), \tag{5.10}$$

where temperature $T > 0$ is in energy units. High T permits exploration; lowering T makes the process more selective. At fixed T, symmetric proposals and this acceptance rule satisfy detailed balance for the distribution proportional to $e^{-E/T}$. A finite cooling schedule need not equilibrate or find the optimum.

Worked example: A controlled uphill move

An uphill change of $\Delta E = 2$ is accepted with probability $e^{-2} \approx 0.135$ at $T = 1$, but only $e^{-4} \approx 0.018$ at $T = 0.5$. This permits escape from local minima early in a run. Random restarts explore other basins; they do not certify optimality.

Ideal quantum annealing instead evolves a Hamiltonian, often

$$H(s) = A(s)H_D + B(s)H_C, \qquad H_D = -\sum_i X_i, \quad s = t/\tau. \tag{5.11}$$

Here τ is the total run time; schedules $A(s), B(s)$ supply consistent energy units and interpolate from driver domination to cost domination. The driver's ground state is $|+\rangle^{\otimes n}$. Since X_i does not commute with diagonal Z costs, the driver permits superpositions of configurations and tunnelling through some barriers.

In a closed, sufficiently smooth evolution with a suitable nonzero spectral gap, the adiabatic theorem bounds excitation when evolution is sufficiently slow. A useful time scale is

$$\tau_{\text{scale}} = \frac{\hbar \max_s |\langle 1(s)|\partial_s H|0(s)\rangle|}{\Delta_{\min}^2}, \tag{5.12}$$

where $\Delta_{\min}$ is the minimum ground-to-excited gap and $|0(s)\rangle, |1(s)\rangle$ are instantaneous ground and excited states. This is intuition, not a universal sufficient runtime formula: rigorous bounds depend on derivatives, endpoints and the chosen error tolerance. Exponentially small gaps can make adiabatic algorithms inefficient.

Real annealers are open, finite-temperature devices. Thermal transitions, calibration, limited connectivity, minor embedding and sampling overhead matter. Tunnelling is a physical mechanism, not proof of a general runtime advantage. SA is entirely classical; calling it quantum-inspired describes its role in this project, not quantum hardware execution.

For independent assignment use matching or min-cost flow; for linear constraints use mixed-integer or constraint programming; for large structured workloads use greedy scheduling and local search. Classical methods can exploit structure that a dense penalty encoding hides. Hybrid solvers can decompose a workload and call different samplers, but benefits must be measured end to end.

Implement a move without recomputing the whole energy. For an upper-triangular QUBO, only terms containing bit i change when it flips. Since the bit increment is $1 - 2x_i$,

$$\Delta_i E = (1 - 2x_i)\left[Q_{ii} + \sum_{j<i} Q_{ji}x_j + \sum_{j>i} Q_{ij}x_j\right]. \tag{5.13}$$

For the allocation state 1001, flipping x_2 gives $\Delta_2 E = -6 + 20 + 10 = 24$, so energy changes from 2 to 26. The constant cancels. This requires only incident nonzero pairs, rather than every pair in Q. Validate this faster update against full recomputation before using it in Athena's annealer; convention errors in the lower/upper indices silently change acceptance probabilities.

5.5 QAOA and hybrid optimisation

QAOA is an existing variational circuit algorithm [10]. For a cost Hamiltonian H_C and mixer $H_M = \sum_i X_i$, define p alternating layers by

$$|\boldsymbol{\gamma}, \boldsymbol{\beta}\rangle = e^{-\mathrm{i}\beta_p H_M} e^{-\mathrm{i}\gamma_p H_C} \cdots e^{-\mathrm{i}\beta_1 H_M} e^{-\mathrm{i}\gamma_1 H_C} |+\rangle^{\otimes n}, \tag{5.14}$$

$$F(\boldsymbol{\gamma}, \boldsymbol{\beta}) = \langle \boldsymbol{\gamma}, \boldsymbol{\beta} | H_C | \boldsymbol{\gamma}, \boldsymbol{\beta} \rangle. \tag{5.15}$$

The $2p$ real parameters control phases and mixing; units are absorbed so the exponents are dimensionless. Our convention minimises energy. The original QAOA literature often maximises a reward, which reverses optimisation signs.

For diagonal H_C, a measurement string x directly supplies $E(x)$. Repeating the circuit S times estimates F by $\hat{F} = S^{-1} \sum_{k=1}^{S} E(x^{(k)})$. A classical optimiser updates parameters. Finally, retain the best *feasible sampled string*, not an expectation masquerading as an assignment.

Worked example: One-qubit cost shows the mechanism

Let $H_C = Z$ and start in $|+\rangle$. Cost evolution rotates the Bloch vector to $(\cos 2\gamma, \sin 2\gamma, 0)$. Mixer evolution $e^{-\mathrm{i}\beta X}$ rotates it about x by 2β, giving

$$F(\gamma, \beta) = \sin(2\beta) \sin(2\gamma).$$

Taking $\gamma = \pi/4$, $\beta = -\pi/4$ yields $F = -1$, the ground energy of Z. This solvable example illustrates alternating noncommuting operations; it supplies no evidence about difficult many-variable instances.

Since diagonal Z_i and $Z_i Z_j$ terms commute, their cost evolution factors into individual phase operations. For a pair term, CNOT–$R_z(2\gamma J)$–CNOT implements $e^{-\mathrm{i}\gamma J Z_i Z_j}$. Dense QUBOs can require many pair gates and routing. Increasing p expands the ideal variational family, because an added zero-angle layer reproduces the old circuit; it does not guarantee that a noisy classical training loop finds improved parameters.

The engineering loop is model → encode → prepare → sample → estimate → update → decode → validate. Constraint-preserving mixers may avoid penalties but require a feasible initial state and more specialised gates. Compare against SA, local search and strong conventional solvers under equal time budgets. No generic QAOA advantage over these baselines follows from the ansatz definition.

Check. If S independent samples have variance v, the estimator variance is v/S. Explain why four times as many samples halves its standard error.

From a two-qubit term to gates. CNOT maps $|a, b\rangle$ to $|a, b \oplus a\rangle$. The intervening $R_z(2\gamma J)$ contributes phase $e^{-\mathrm{i}\gamma J(-1)^{a \oplus b}}$; a second CNOT restores b. Since $(-1)^{a \oplus b} = (-1)^a (-1)^b$, this is precisely the phase of $e^{-\mathrm{i}\gamma J Z_1 Z_2}$. Check the four phases on $00, 01, 10, 11$: they are $e^{-\mathrm{i}\gamma J}, e^{\mathrm{i}\gamma J}, e^{\mathrm{i}\gamma J}, e^{-\mathrm{i}\gamma J}$. This basis-state test connects Hamiltonian algebra to a directly implementable subcircuit.

6 Quantum machine learning

6.1 Variational models and a learning objective

Source trail: [5], *Codebook variational module and parameter-shift documentation;* [6]. *References on p. 40.*

A variational quantum circuit is a differentiable family of unitaries $U(\boldsymbol{\theta})$, where the real vector $\boldsymbol{\theta}$ contains trainable gate angles. A classical computer chooses the parameters; a quantum processor or simulator prepares states and estimates observables. "Quantum neural network" usually names such a parameterised model, not a direct quantum copy of a brain or of a classical large language model.

For classical input vector z, a feature map $V(z)$ prepares $|\phi(z)\rangle = V(z)|0\rangle^{\otimes n}$. Define a scalar model

$$f_{\boldsymbol{\theta}}(z) = \langle \phi(z)|U(\boldsymbol{\theta})^{\dagger} O U(\boldsymbol{\theta})|\phi(z)\rangle, \tag{6.1}$$

where Hermitian O is the measured observable. If $O = Z_1$, outputs lie in $[-1, 1]$. With labelled data (z_k, y_k), $k = 1, \ldots, K$, a simple objective is

$$L(\boldsymbol{\theta}) = \frac{1}{K}\sum_{k=1}^{K} (f_{\boldsymbol{\theta}}(z_k) - y_k)^2 + \lambda \sum_j \theta_j^2. \tag{6.2}$$

The nonnegative λ controls regularisation. Angle periodicity means this particular penalty depends on the chosen parameter representation; it is an engineering choice, not a mandatory quantum rule. Classification can predict a sign, or use $p(y = 1 \mid z) = (1 + f(z))/2$ as a binary probability model.

Worked example: A fully solvable one-qubit model

Choose $V(z) = R_y(z)$, $U(\theta) = R_y(\theta)$, and $O = Z$. Rotations about the same axis add, giving $f_\theta(z) = \cos(z + \theta)$. For $z = 0$, $y = 1$, and no regularisation, the loss is $(\cos\theta - 1)^2$. At $\theta = \pi/2$, the prediction is zero and the loss is one. At $\theta = 0$, it predicts one and the loss vanishes.

This model is easily simulated classically; a quantum device adds no computational advantage. Its value is pedagogical: every feature, parameter, expectation and loss can be calculated on paper. Adding entangling gates and more layers changes the function family, but expressiveness alone says nothing about generalisation or trainability.

A learning pipeline needs train/validation/test separation, suitable input scaling, a prediction metric and a strong classical comparator. Avoid leaking the same workflow instance or task template across splits. For Athena, task latency or failure prediction is a possible application, but linear models, trees and classical neural networks are natural starting points.

Check. With $z = \pi/3$ and $\theta = 0$, the model gives $f = 1/2$ and class-one probability $3/4$.

Connect loss to a parameter update. For the one-qubit example and one data point, write $L(\theta) = [\cos(z + \theta) - y]^2$. Differentiating the outer square first gives

$$L'(\theta) = -2[\cos(z + \theta) - y]\sin(z + \theta).$$

For $z = 0, y = 1, \theta = \pi/3$, the gradient is $\sqrt{3}/2$. With learning rate 0.1, gradient descent moves to approximately 0.9606 radians. Predict the direction before calculating: reducing a positive angle toward zero raises $\cos\theta$ toward its target one. Chapter 6.2 obtains the same circuit derivative from measurable shifted expectations, so the classical update rule does not depend on how that derivative was computed.

6.2 Deriving the parameter-shift gradient

Source trail: [5], *parameter-shift documentation;* [11]. *References on p. 40.*

Consider a parameter occurring in one gate $R_P(\theta) = e^{-\mathrm{i}\theta P/2}$ with $P^2 = I$, surrounded by parameter-independent gates. Holding every other parameter fixed, insert $R_P = \cos(\theta/2)I - \mathrm{i}\sin(\theta/2)P$ into an expectation. Products of sine and cosine reduce it to

$$f(\theta) = a + b\cos\theta + c\sin\theta, \tag{6.3}$$

where real a, b, c depend on the other gates, input state and observable. Differentiating gives $f'(\theta) = -b\sin\theta + c\cos\theta$. Evaluating at shifted angles and subtracting yields the exact rule

$$\frac{\partial f}{\partial \theta} = \frac{f(\theta + \pi/2) - f(\theta - \pi/2)}{2}. \tag{6.4}$$

It is an identity for exact expectations, not a small-step finite difference. Shot estimates of those expectations are still noisy. Other generators need shift rules determined by their eigenvalue gaps. If one parameter is reused in several gates, differentiate each occurrence and sum with the chain rule; blindly shifting the shared parameter once can fail.

Worked example: Gradient and one classical update

For $f_\theta(0) = \cos\theta$ at $\theta = \pi/2$, Eq. (6.4) gives $[\cos\pi - \cos 0]/2 = -1$. With target one and $L = (f-1)^2$, the chain rule gives $L' = 2(f-1)f' = 2$. Gradient descent with step $\eta = 0.1$ updates $\theta \leftarrow \pi/2 - 0.2$. The new prediction is $\sin 0.2 \approx 0.1987$, and the loss falls to about 0.642.

For the data objective, the gradient is

$$\partial_j L = \frac{2}{K}\sum_k (f_{\boldsymbol{\theta}}(z_k) - y_k)\,\partial_j f_{\boldsymbol{\theta}}(z_k) + 2\lambda\theta_j. \tag{6.5}$$

One must apply the chain rule to the classical loss. Shifting the whole squared loss generally does not obey Eq. (6.4), because squaring introduces additional frequencies.

For a ± 1 observable, S independent shots give expectation variance at most $1/S$. With independent shifted estimates, the gradient variance is at most $1/(2S)$. Achieving standard error ϵ therefore needs order $1/\epsilon^2$ shots in this basic estimator. For P independently occurring parameters, a full gradient requires $2P$ shifted circuits per data item and measurement setting, before batching or other estimator improvements.

Check. For $f(\theta) = \cos\theta$ and $\theta = \pi/3$, derive $f' = -\sqrt{3}/2$ with both differentiation and shifts. Distinguish exact differentiation from finite-shot estimation.

Expose the trigonometry. Writing $c = \cos(\theta/2)$ and $s = \sin(\theta/2)$, the expectation contains terms proportional to c^2, s^2 and cs. Use $c^2 = (1 + \cos\theta)/2$, $s^2 = (1 - \cos\theta)/2$ and $cs = \sin\theta/2$ to obtain $a + b\cos\theta + c_1\sin\theta$ with fixed coefficients. Then $\cos(\theta + \pi/2) - \cos(\theta - \pi/2) = -2\sin\theta$ and the analogous sine difference is $2\cos\theta$. Dividing their linear combination by two gives the derivative. The symbol c_1 here is a coefficient, distinct from the temporary cosine c.

6.3 Quantum feature maps and kernels

A kernel compares inputs through an implicit feature space. Given normalised states $|\phi(z)\rangle$, a common quantum kernel is

$$k(z, z') = |\langle\phi(z)|\phi(z')\rangle|^2. \tag{6.6}$$

It is a positive semidefinite kernel because it equals the Hilbert–Schmidt inner product $\mathrm{Tr}(\rho_z \rho_{z'})$ for $\rho_z = |\phi(z)\rangle\langle\phi(z)|$. For real coefficients v_i,

$$\sum_{ij} v_i v_j k(z_i, z_j) = \mathrm{Tr}\left[\left(\sum_i v_i \rho_{z_i}\right)^2\right] \geq 0. \tag{6.7}$$

This positivity supports conventional kernel learning. The quantum part estimates entries; the optimisation of classifier coefficients may be entirely classical.

To estimate an entry, prepare $V(z')^\dagger V(z)|0\rangle$ and measure the probability of all zero. Its amplitude is $\langle 0|V(z')^\dagger V(z)|0\rangle = \langle\phi(z')|\phi(z)\rangle$. The order of preparation and inverse matters.

Worked example: An elementary kernel has an elementary classical formula

With $|\phi(z)\rangle = R_y(z)|0\rangle$,

$$k(z, z') = \cos^2((z - z')/2) = \frac{1 + \cos(z - z')}{2}.$$

For inputs 0 and $\pi/2$, their Gram matrix is $\begin{pmatrix} 1 & 1/2 \\ 1/2 & 1 \end{pmatrix}$, with eigenvalues $3/2$ and $1/2$. The feature map is valid, but no quantum processor is needed to calculate it.

For K training points, a dense symmetric Gram matrix has $K(K-1)/2$ off-diagonal entries. Estimation to additive precision ϵ by basic repeated sampling costs order $1/\epsilon^2$ shots per entry for fixed confidence, and logarithmic confidence factors may be needed for many entries. Preparation, communication and training time must all be counted. Noise can make an estimated Gram matrix indefinite; symmetrising or projecting onto the positive cone changes the model and must be reported.

A feature map difficult to simulate classically does not automatically produce a useful kernel. Similarities might concentrate near one value, losing discrimination. A model can be expressive yet poorly matched to the data. Compare with linear, polynomial and radial-basis kernels and with task-specific classical features.

For Athena, possible features include task duration estimates, capability requirements, context reuse and dependency summaries. The embedding is a proposed engineering choice. Neither the kernel formula nor high-dimensional Hilbert space proves better prediction or scheduling. Data-loading cost is especially important when inputs already live in classical memory.

Check. Show $k(z, z) = 1$ for every normalised preparation, and explain why amplitude-encoding an arbitrary length-2^n vector may itself require many operations.

6.4 Trainability, barren plateaus and honest limits

Source trail: [6], *optimisation loops;* [12]. *References on p. 40.*

A barren plateau is a training regime in which gradient variance shrinks exponentially with qubit number under specified circuit and cost assumptions. In sufficiently random expressive circuits, concentration of measure can make expectation values almost constant across parameter space. This is not a theorem that every quantum model is untrainable, nor does a nonzero analytic gradient guarantee it is measurable with a realistic shot budget.

If a typical gradient magnitude is order $2^{-n/2}$ while shot error scales as $S^{-1/2}$, resolving it at comparable error can require S of order 2^n. Exact parameter-shift formulas do not remove this statistical cost. Noise may suppress signal further. Gradient-free optimisers also need a distinguishable cost signal and are not a universal cure.

Practical design responses include shallower circuits, local observables, structured initial parameters and problem-informed ansatze. They are methods to test, not general guarantees. Increasing depth can improve representational capacity while worsening noise and optimisation. Measure the tradeoff rather than assuming bigger is better.

Worked example: A shot-budget sanity check

Suppose an observable has standard deviation near one and two parameter choices differ in expectation by 10^{-3}. With $S = 10^4$ independent shots the standard error is roughly 10^{-2}, larger than the difference. Order 10^6 shots are needed for a single-estimate error of 10^{-3}, and reliable comparison can require more. A small numerical loss change may be indistinguishable experimentally.

A sound hybrid learning loop is:

1. Fix data splits, input scaling, circuit family and measurement budget.
2. Estimate outputs and, when useful, gradients on training batches.
3. Update parameters with a classical optimiser and record total costs.
4. Select settings using validation data, then evaluate once on held-out tests.
5. Repeat across seeds; compare with classical models at matched budgets.

Quantum learning can have proven advantages for carefully specified access models or quantum-generated data. These do not establish a broad practical advantage for classical text, ordinary tabular data or Athena's intelligence. Claims require stating data access, output requirements, precision, sample complexity and the best known classical alternative.

Research boundary. A variational circuit might be a useful component of an Athena prediction pipeline; this remains a hypothesis. Solving a QUBO for task allocation improves a decision policy only to the extent that the cost model captures useful real outcomes. It does not by itself improve reasoning, factual accuracy or safe execution.

Check. Design an ablation that keeps the classical optimiser and data fixed while replacing the quantum model by a classically tractable feature map. What evidence would attribute an improvement to the quantum component?

7 Quantum cryptography

7.1 BB84: preparation, sifting and disturbance

Source trail: [7], *Week 6;* [2], *Lecture 31. References on p. 40.*

Quantum key distribution (QKD) uses a quantum channel plus an authenticated classical channel to establish shared secret key material. It requires devices that prepare and measure quantum states; ordinary classical laptops alone cannot implement its physical security mechanism. A classical simulation illustrates probabilities, not secure physical distribution.

In ideal BB84, Alice chooses a random bit a and random basis $b \in \{0, 1\}$. She sends $\mathsf{H}^b|a\rangle$. Thus the four states are $|0\rangle, |1\rangle, |+\rangle, |-\rangle$. Bob independently chooses a basis b' and measures. Equal bases reproduce Alice's bit ideally; different bases produce a uniformly random bit because $|\langle 0 \mid +\rangle|^2 = |\langle 1 \mid +\rangle|^2 = 1/2$ and similarly for minus.

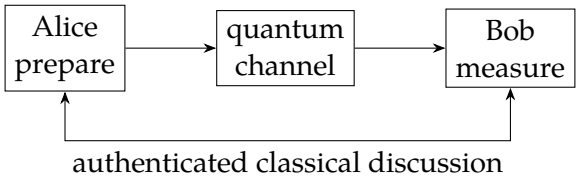
After receipt, Alice and Bob announce bases over the authenticated classical channel and discard mismatches. This sifting retains about half the transmissions for uniform independent basis choices. They reveal a randomly chosen subset of retained bits to estimate the quantum bit error rate (QBER). The revealed test bits are removed from potential key material.

Worked example: Intercept-and-resend produces a 25% sifted error rate

Suppose Eve intercepts every signal, measures in a random BB84 basis, and resends the state she found. Conditional on Alice and Bob using the same basis, Eve chooses the wrong basis with probability $1/2$. In that branch Bob disagrees with Alice with probability $1/2$. Hence the sifted QBER is $(1/2)(1/2) = 1/4$. In an ideal independent model, m tested sifted bits all avoid an error with probability $(3/4)^m$.

This calculation demonstrates one attack, not unconditional security. Eve may keep quantum memory, entangle signals with ancillas, and wait for public announcements. A security proof must bound her information under the specified device and channel assumptions.

No-cloning rules out making perfect copies of all four possible signal states. It does not forbid copying known orthogonal states, nor does it show that every measurement necessarily disturbs every input. Security depends on incompatible possible preparations, statistical testing and post-processing together.



Check. If Eve attacks only a fraction f of ideal signals in this way, the expected sifted QBER contribution is $f/4$. Background noise makes observed errors an imperfect indicator of the attack mechanism.

Calculation exercise. In the independent full intercept-resend model above, find the smallest tested sample with probability below 1% of seeing no errors. Solve $(3/4)^m < 0.01$: $m > \log(0.01)/\log(3/4) \approx 16.01$, so $m = 17$. Check: $(3/4)^{16} \approx 0.01002$, but $(3/4)^{17} \approx 0.00752$. This is an attack-specific calculation, not a finite-key security proof.

7.2 From raw correlations to a secure key

Sifted bits are not automatically secret or equal. Information reconciliation corrects disagreements using classical messages that leak some information. Privacy amplification then compresses the reconciled string using a suitable randomly chosen hash function to reduce Eve's residual information. Authentication prevents an active adversary from replacing the classical discussion with separate conversations.

A finite-key analysis accounts for parameter-estimation uncertainty, reconciliation leakage, imperfect devices and a chosen failure tolerance. No observed errors in a small test sample does not prove zero error or perfect secrecy in the remaining bits. A security model should allow Eve all computations permitted by quantum mechanics and whatever device interactions the model admits, rather than only the intercept-resend attack.

One precise target uses a joint classical-quantum state ρ_{KE} of the key K and Eve's system E. Conditional on not aborting, secrecy asks for closeness to a uniform key independent of Eve, measured through a trace-distance expression such as

$$p_{\text{pass}} \frac{1}{2} \left\| \rho_{KE|\text{pass}} - \tau_K \otimes \rho_{E|\text{pass}} \right\|_1 \leq \epsilon_{\text{sec}}. \tag{7.1}$$

Here p_{pass} is the probability of not aborting, τ_K is the uniform key state and $\|A\|_1 = \operatorname{Tr} \sqrt{A^\dagger A}$ is the trace norm. The weighting handles protocols that often abort. Correctness separately bounds the probability that Alice and Bob output different keys while passing, by ϵ_{cor}. These bounds express limited distinguishability from an ideal key resource, not metaphysical certainty.

Worked example: Why a classical channel needs authentication

If Eve can impersonate both parties, she can run one QKD session with Alice and another with Bob. Each session may have low error. Alice and Bob nevertheless share keys with Eve, not with one another. An initial authentication resource, for example a small shared secret or an appropriate signature scheme under its assumptions, prevents this substitution.

Practical threats include multi-photon emissions, detector behaviour, side channels, randomness failure and implementation faults. Device-independent approaches use different assumptions and demanding experimental tests; they do not remove every engineering assumption. A full proof is beyond this chapter, but the protocol steps and the scope of the claim must remain explicit.

QKD distributes keys; it does not itself authorise agents, secure operating systems or protect stored plaintext. Authentication keys and refresh policies still need lifecycle management. Athena's remote devices would need suitable quantum links and endpoints for QKD. For ordinary internet connectivity, conventional authenticated encryption and post-quantum migration are the more directly applicable engineering topics.

Check. Explain why privacy amplification must account for reconciliation messages even when those messages travel over an authenticated channel. Authentication prevents tampering; it does not imply confidentiality.

7.3 Shor's threat and post-quantum engineering

Source trail: [1], *Lesson 7;* [13], *NIST standards and migration FAQ. References on p. 40.*

Shor-type algorithms solve integer factoring and discrete logarithms in polynomial quantum time. Factoring undermines RSA's mathematical foundation; discrete-log algorithms undermine conventional finite-field Diffie–Hellman and elliptic-curve public-key schemes. This statement assumes a sufficiently capable quantum computer; it is not a claim that an arbitrary current processor can break deployed keys.

Grover's search has a different implication. It offers a generic quadratic query improvement for brute-force search, with coherence, oracle and parallel-resource requirements. It does not produce Shor's polynomial-time transformation for every symmetric primitive. Exact engineering security levels require analysing the actual construction and attack costs.

	Quantum cryptography	Post-quantum cryptography (PQC)
Mechanism	Quantum states and measurement, as in QKD	Classical algorithms based on problems believed hard for quantum attackers
Hardware	Quantum communication endpoints and channel	Ordinary classical processors and networks
Assumptions	Specified device, channel and authentication model	Computational hardness and correct implementation
Athena relevance	Specialised link deployment	Key establishment, signatures and migration planning

NIST published FIPS 203 (ML-KEM), 204 (ML-DSA) and 205 (SLH-DSA) in 2024 [13]. ML-KEM is a key-encapsulation mechanism for establishing shared secrets; the latter two are signature standards. These are not bulk data-encryption algorithms. Their design targets quantum-resistant security under their assumptions, not a proof that every possible future attack is impossible.

Worked example: Inventory before migration

Suppose Athena uses TLS to reach model services, SSH for a development host, signed agent updates and encrypted backups. Record the key-establishment and signature mechanisms, libraries, protocol support and expected confidentiality lifetime. A backup encrypted with a symmetric key may still depend on a vulnerable public-key wrapping mechanism. Long-lived captured traffic creates a "harvest now, decrypt later" concern if its key exchange is later broken.

The proposed engineering application is crypto-agility: record algorithm dependencies, use maintained cryptographic libraries, support tested upgrades and track protocol interoperability. Do not invent a custom lattice scheme or treat a quantum simulator as a security feature. Key management, least privilege, signed updates, audit logs and protection of execution credentials remain central for an autonomous assistant.

Check. Distinguish a signature from key encapsulation and symmetric encryption. A signed agent update proves authenticity under the signature assumptions; it does not hide its contents or prove that its code is safe.

8 Athena: advanced algorithm research

8.1 A decision model before a solver

Source trail: Established encoding methods: [4]; existing variational methods: [6,10]. Athena design below is proposed.. References on p. 40.

The **Athena Quantum-Inspired Optimisation Engine** is a proposed decision component. It accepts tasks, agents, capabilities, predicted costs and constraints; returns a validated plan; and records why a solver chose it. Its first implementation should be classical. The term quantum-inspired here means using an energy-model interface compatible with annealing and optional quantum experiments. SA itself is a classical method.

Let $\mathcal{T}$ be a set of tasks and $\mathcal{A}$ a set of available agents or execution configurations. For task i and agent a, let c_{ia} be monetary cost, d_{ia} predicted duration, e_{ia} energy, q_{ia} predicted failure probability, and $k_{ia} \in \{0, 1\}$ capability eligibility. Let x_{ia} indicate assignment. A simple cost is

$$f(x) = \sum_{i,a} w_{ia} x_{ia} + \sum_{i<j,a,b} r_{iajb} x_{ia} x_{jb}, \tag{8.1}$$

where $w_{ia} = \alpha c_{ia} + \beta d_{ia} + \gamma e_{ia} + \delta q_{ia}$. Nonnegative weights convert documented, preferably normalised units into one score. Pair costs r_{iajb} can represent communication overhead, resource conflict or a negative benefit for reusable context. They must be estimated rather than assumed beneficial.

Require $\sum_a x_{ia} = 1$ and $x_{ia} = 0$ when $k_{ia} = 0$. For a one-batch capacity model, require $\sum_i u_{ia} x_{ia} \leq B_a$, with integer resource demand u_{ia} and capacity B_a. This is not yet a full scheduler: a sum of durations does not model parallel makespan, and an assignment does not specify start times.

Worked example: Model selection as an assignment dimension

Treat "agent A with small local model" and "agent A with large remote model" as separate configurations. Each has different price, latency and eligibility. Add a mutual resource constraint if both share the same physical device. Without it, the optimiser might schedule impossible simultaneous work even though each configuration looks individually valid.

An autonomous workflow is a directed acyclic graph (DAG): an edge (i, j) requires task i to finish before j begins. A topological order exists for a DAG; cycles indicate inconsistent prerequisites or a need to model iteration explicitly. Ready-task batching is a useful simplified first prototype: solve only for tasks whose predecessors are already complete, execute the batch, then replan.

Evidence status. Binary quadratic encoding and penalty bounds are established mathematics. SA, matching and mixed-integer optimisation are existing algorithms. Integrating them into Athena is a proposed application. Better cost, latency or intelligence is a hypothesis requiring measured workload outcomes.

A concrete greedy failure. Task 1 may use A at cost 1 or B at cost 2; task 2 may use only A at cost 1. Both agents have unit capacity. Processing task 1 first with cheapest-agent selection consumes A and blocks task 2. The feasible optimum is task 1 on B and task 2 on A, cost 3. Eligibility is not merely another soft preference: dropping it creates invalid plans. Processing the most constrained task first fixes this instance, but coupled costs still require a broader comparison.

8.2 Dependency-aware scheduling and QUBO conversion

For a full discrete-time model let $t \in \{0, \dots, H-1\}$ index time slots and let $y_{iat} = 1$ mean task i starts on agent a at t. Here H is a planning horizon, unrelated to the Hamiltonian notation. Remove triples with ineligible capabilities, starts before release time r_i, completion beyond the horizon H, or completion after a hard deadline D_i. Durations d_{ia} are positive integers in slots. Define the valid-start set $\mathcal{U}_i$ and require

$$\sum_{(a,t)\in\mathcal{U}_i} y_{iat} = 1 \quad \text{for each } i. \tag{8.2}$$

For unary-capacity agents, forbid pairs of distinct tasks on the same agent whose half-open execution intervals $[t, t + d_{ia})$ overlap. For each dependency edge (i, j), forbid pairs with $t_i + d_{ia} > t_j$. Each forbidden pair can be penalised by the product of its two start bits. Thus an exact discrete model is a QUBO,

$$E(y) = \sum_{i,a,t} g_{iat} y_{iat} + A \sum_i \left(\sum_{(a,t)\in\mathcal{U}_i} y_{iat} - 1 \right)^2 + A \sum_{\{u,v\}\in\mathcal{F}} y_u y_v. \tag{8.3}$$

Here g_{iat} is a documented start-dependent cost, u, v index valid triples, and $\mathcal{F}$ is the set of unordered forbidden pairs, counted once. It includes agent overlap and precedence violations. A soft deadline can instead add $\lambda_D \max(0, t + d_{ia} - D_i)$ to g_{iat}. Keep hard and soft deadlines distinct.

For higher capacities or shared memory/GPU demands, use occupancy inequalities per slot and resource, with integer slack encoding as in Chapter 5. A makespan objective needs an additional completion-time variable or an explicit horizon feasibility search; a sum of completion times is a different objective.

Worked example: A two-task dependency fixes the valid order

Tasks 1 and 2 each take one slot, with edge $1 \to 2$, one agent, and $H = 2$. Both initially have candidate starts 0 and 1. The sole feasible selection is $y_{1,A,0} = y_{2,A,1} = 1$. Selecting both at 0 violates overlap and precedence; selecting task 1 at 1 and task 2 at 0 violates precedence. Completion at time 2 is allowed because the horizon contains slots 0 and 1.

The number of start variables can grow as $|\mathcal{J}||\mathcal{A}|H$, with many pair couplings. A smaller time slot improves temporal fidelity but increases model size. This is a concrete reason to compare direct constraint programming with QUBO, rather than assuming the encoding is the best representation.

To make penalty selection rigorous, bound the range of the unpenalised finite objective by $R = \sum_u |g_u| + \sum_{u<v} |r_{uv}|$ when extra pair costs are present. If a feasible plan exists, $A > R$ excludes every violating minimiser, since any violation adds at least A. Validate feasibility again after decoding; an approximate solver need not find the true minimum.

Write the toy scheduling energy explicitly. Let $u = y_{1,A,0}$, $v = y_{1,A,1}$, $w = y_{2,A,0}$, $z = y_{2,A,1}$. The forbidden unordered pairs are uw, vw, vz. With zero base costs,

$$E = A[(u + v - 1)^2 + (w + z - 1)^2 + uw + vw + vz].$$

Its expansion has constant $2A$, every diagonal $-A$, one-hot pair coefficients $2A$ on uv, wz, and coefficients A on uw, vw, vz. Only $(u, v, w, z) = (1, 0, 0, 1)$ has zero energy. Enumerating 16 configurations tests the scheduling compiler separately from its solver. Deduplicating a pair that violates both overlap and precedence keeps its coefficient at A under the set convention in Eq. (8.3).

8.3 Prototype pseudocode and a reproducible small experiment

The minimal executable design uses ready tasks and unary batch capacity, reproducing Chapter 5 before tackling Eq. (8.3). Inputs include task IDs, allowed agent IDs, a nonnegative cost table and a seed. Output includes assignment, original cost, feasibility, energy, timing and settings. The following Python-style pseudocode deliberately separates model correctness from solver quality.

```
def energy(x, Q, offset):
    return offset + sum(Q[i,j]*x[i]*x[j]
                        for i,j in Q)  # upper triangle only
def exact(Q, offset, n):
    # Enumerate 2**n strings: validation baseline, not scalable.
    return min(all_bitstrings(n),
               key=lambda x: energy(x, Q, offset))
def anneal(Q, offset, n, temperatures, sweeps, seed):
    rng = Random(seed)
    x = random_bits(n, rng)
    best = x.copy()
    for T in temperatures:      # T > 0; fixed in advance
        for sweep in range(sweeps):
            for i in shuffled(range(n), rng):
                candidate = x.copy()
                candidate[i] ^= 1
                delta = energy(candidate,Q,offset)-energy(x,Q,offset)
                if delta <= 0 or rng.random() < exp(-delta/T):
                    x = candidate
                if energy(x,Q,offset) < energy(best,Q,offset):
                    best = x.copy()
    return best
def compare(tasks, agents, costs, config):
    start = monotonic_clock()
    Q, offset, decode = build_batch_qubo(tasks,agents,costs)
    build_time = monotonic_clock() - start
    assert_encoding_on_tiny_cases(Q, offset, tasks, agents)
    for name, solve in solver_adapters(tasks,agents,costs,Q,
                                       offset,config):
        start = monotonic_clock()
        raw = solve()  # backend setup and search included
        if raw is None:
            record(name, status="no_solution",
                   seconds=build_time+monotonic_clock()-start)
            continue
        result = decode(raw)
        valid = check_all_constraints(result)
        cost = original_cost(result) if valid else None
        record(name, result, valid, cost,
               build_time+monotonic_clock()-start)
    # Optional backend: same model; include queue/sample time.
```

Adapters bind backend inputs and return bits or None; backend setup is timed. Charge shared build time to each method; tiny-case assertions are development checks. Greedy assigns tasks in a declared order to the cheapest eligible unused agent and reports failure if stuck. Also compare optimal matching: greedy can fail even when a feasible assignment exists.

For the numerical Q in Chapter 5, exact enumeration must return 1001 with energy two, using our bit order. A claimed different minimum signals an implementation or convention error. The optional companion script implements this check and SA with fixed seeds; the PDF calculation alone is sufficient to reproduce the conclusion. SA outcomes and wall-clock times must be measured, not assumed.

8.4 Evaluation, extensions and research hypotheses

A good optimiser minimises operational cost while returning feasible plans within a decision deadline. Solver quality must be evaluated with the real business objective, not only its internal penalty energy. Fix instance families, budgets, hardware and stopping rules before comparing methods.

Use exact enumeration on tiny cases to validate encodings; use matching for pure assignment and a conventional mixed-integer or constraint solver for richer models. Add greedy plus local search as a practical baseline and SA as the energy-based candidate. An optional annealer or QAOA backend receives the same instances, with its embedding, parameter search and all repetitions included in the budget.

Metric	Operational interpretation
Feasibility rate	Fraction satisfying every original hard constraint
Cost gap	$(f - f^*)/\max(1, \lvert f^* \rvert)$ when an optimum f^* is known
Decision latency	Model construction, solve, decode, validate and repair time
Workflow outcomes	Makespan, deadline misses, spend, failures and energy estimate
Robustness	Performance across seeds, prediction errors and workload shifts

For independent trials with success probability p and duration t, the number needed for at least success probability q is $\lceil \log(1-q)/\log(1-p) \rceil$, for $0 < p, q < 1$. Time to target is this number times t, plus unavoidable setup overhead. This assumes independent, comparable trials; adaptive training runs may violate it. Report confidence intervals for estimated p and handle zero observed successes without pretending the rate is exactly zero.

Proposed improvements. Warm-start from the previous feasible plan; decompose large DAGs into ready-task windows; preserve scarce agent capacity; estimate duration uncertainty; and replan after observed completion. Use chance constraints or sampled scenarios when deadlines depend on uncertain times. Penalise unnecessary reassignment to avoid planning churn. A fast incumbent can preserve execution continuity while a slower optimiser searches.

Research hypotheses. SA with swap moves may find better equal-time assignments than a fixed greedy order when pairwise context costs dominate. Structured QAOA may improve samples on selected instance families. Neither proposition follows from the encoding; each needs experiments. Compare swap-based local search to isolate the benefit of annealing itself.

Learning w_{ia} from logs is another research direction: predicted cost accuracy may matter more than solver sophistication. Evaluate frozen predictions and solver changes separately. Test with recorded or synthetic workflows before granting a proposed plan permission to execute external actions. The optimiser chooses among eligible actions; it should not define the eligibility policy.

Check. If a method has better raw QUBO energy but more constraint violations, can it be called a better scheduler? Explain using the feasibility and business-cost metrics.

9 Exercises and research challenges

9.1 Derivation problems with answer checks

Attempt each problem before reading its check. A correct numerical answer is only the first step: explain the assumptions and derive the expression you used.

1. **Complex linear algebra.** Let $u = (1, \mathrm{i})^T/\sqrt{2}$ and $v = (\mathrm{i}, 1)^T/\sqrt{2}$. Compute $u^\dagger v$, $uu^\dagger$ and $\mathrm{Tr}[(uu^\dagger)^2]$.
   Check: The overlap is zero; $uu^\dagger = \frac{1}{2}\begin{pmatrix} 1 & -\mathrm{i} \\ \mathrm{i} & 1 \end{pmatrix}$; purity is one. Explain why using u^T instead of $u^\dagger$ fails.

2. **Eigenstructure and evolution.** Diagonalise $H = \hbar\omega X/2$. Starting from $|0\rangle$, find $\Pr(1)$ at time t.
   Check: Eigenstates are $|\pm\rangle$, energies $\pm\hbar\omega/2$. Evolution gives $\cos(\omega t/2)|0\rangle - \mathrm{i}\sin(\omega t/2)|1\rangle$; probability is $\sin^2(\omega t/2)$.

3. **Tensor products and entanglement.** Expand $(|0\rangle + \mathrm{i}|1\rangle) \otimes (|0\rangle - |1\rangle)/2$. Apply CNOT. Determine whether the output factors.
   Check: Input amplitudes are $(1, -1, \mathrm{i}, -\mathrm{i})/2$; output is $(1, -1, -\mathrm{i}, \mathrm{i})/2$. Its determinant test is zero: it remains a product. CNOT does not entangle every product input.

4. **Mixed states and gates.** Compare applying H to ρ_+ and $I/2$. Derive output matrices and Z probabilities.
   Check: $\mathsf{H}\rho_+\mathsf{H} = |0\rangle\langle 0|$, whereas $\mathsf{H}(I/2)\mathsf{H} = I/2$. Coherence enables deterministic interference.

5. **Grover.** For $N = 16, M = 1$, calculate p_r at $r = 2, 3, 4$. Explain why a fourth iteration is worse than a third.
   Check: $\theta = \arcsin(1/4)$; probabilities are approximately $0.9084, 0.9613, 0.5817$. The state overshoots the good direction. Query order is $\sqrt{N}$, not $\log N$.

6. **QFT.** Compute $F_4(|0\rangle - |2\rangle)/\sqrt{2}$. Apply $F_4^\dagger$ to your result.
   Check: The result is $(|1\rangle + |3\rangle)/\sqrt{2}$; the inverse returns the input. Even output indices cancel in the forward transform.

7. **QUBO.** Minimise $2x_1 + x_2$ subject to $x_1 + x_2 = 1$. Use penalty $A = 4$, state Q, C and enumerate all energies.
   Solution: Expand $4(x_1+x_2-1)^2 = 4-4x_1-4x_2+8x_1x_2$. Add $2x_1+x_2$ to obtain $E = 4-2x_1-3x_2+8x_1x_2$. Thus $Q = \begin{pmatrix} -2 & 8 \\ 0 & -3 \end{pmatrix}$ and $C = 4$. Substitute each string: $E(00) = 4$, $E(01) = 4 - 3 = 1$, $E(10) = 4 - 2 = 2$, and $E(11) = 4 - 2 - 3 + 8 = 7$. Only $01, 10$ satisfy the original constraint; comparing their unpenalised costs, $1 < 2$, confirms optimum 01 independently of Q.

8. **Gradient.** For $f(\theta) = \cos\theta$ and $L = (f + 1)^2$, compute L' at $\theta = \pi/2$.
   Check: $f' = -1$ and $L' = -2$. Explain why shifting L itself is not the parameter-shift identity for f.

Solution workshop for Problem 6. Form $v = (1, 0, -1, 0)^T/\sqrt{2}$. Row multiplication gives

$$F_4 v = \frac{1}{2\sqrt{2}}(1-1,\ 1+1,\ 1-1,\ 1+1)^T = (0, 1, 0, 1)^T/\sqrt{2}.$$

For the inverse, conjugate F_4 and add its columns 1 and 3: their sum is $(2, 0, -2, 0)^T/2$. Dividing by $\sqrt{2}$ returns v. This verifies reversibility on a superposition, not only on basis inputs.

9.2 Mini research challenge: can Athena allocate better?

Question. Can a quantum-inspired optimiser improve Athena's task allocation compared with a conventional scheduling algorithm? Formulate a narrow, falsifiable claim before implementing a backend.

Hypothesis. At a fixed end-to-end decision budget, SA with bit and feasible assignment-swap moves reduces median feasible objective by at least 5% relative to greedy plus local search on held-out instances with strong pairwise context costs, while maintaining at least 99% feasibility. These are proposed targets, not results. A negative result is useful if the experiment can detect the intended effect.

Instance design. Generate task counts $8, 16, 32, 64$, several agent counts, capability sparsity, capacity limits, dependency density and pairwise-cost strengths. Construct feasible instances from a known plan, then vary durations and costs. Separate assignment-only and scheduling families. Add recorded workflows only when cost and duration definitions are consistent. Save every instance and generator seed.

Baselines. Include exact enumeration for tiny encodings, optimal matching for independent unit-capacity assignment, a conventional constraint or mixed-integer solver for scheduling, greedy with a documented order, and greedy plus local search. The candidate is SA; optional quantum runs are a separate arm. An exact solver can use its incumbent under the same timeout, while reporting an optimality bound when available.

Variables and controls. Vary size, eligibility, interaction strength, prediction error and budget. Tune temperature schedules, penalties and move types on training instances only. Hold out entire instance families or workflow templates. Use multiple predeclared seeds, for example 30 per instance, and budgets including model construction, decoding, repair and validation. Record failures instead of deleting them.

Metrics and analysis. Report feasibility first, then cost gaps, decision-time distributions, deadline misses, makespan and spend. Compare methods on the same instances. Bootstrap paired per-instance differences to obtain a 95% interval; average repeated seeds within each instance first to avoid treating correlated repetitions as independent workloads. State the resampling method and show failures. Report effect sizes and distributions, not only the best run.

Ablations. Remove pair costs; replace cooling with fixed-temperature search; replace bit moves with swaps; warm-start methods equally. Separate benefits from a better model, starting plan and extra compute. For quantum runs include network and queue latency, embedding, calibration, parameter tuning and all shots; also preserve device-only timing for diagnosis.

Decision rule. Adopt the candidate only if the predefined quality target holds on held-out workloads within the deadline and feasibility requirement. Otherwise retain the conventional method and document where each solver works. A successful SA result establishes a classical engineering improvement, not quantum advantage or improved general intelligence.

Next paper exercise. Design a four-task instance where the cheapest eligible agent for the first task is a later task's only feasible agent. Solve it manually and identify the information missing from the greedy rule.

References and further study

Sources support established concepts; exposition, examples, diagrams and the Athena design are original. Access checked 18 October 2026. Reading was selective, not a claim to have read every linked course. The companion source map records acquisition limits. These links are optional further study.

[1] J. Watrous, *Understanding Quantum Information and Computation* (2025), Lessons 1–3, 5–9, 11, 13–14. arXiv:2507.11536. Full PDF downloaded for reference.

[2] P. Shor, MIT 8.370/18.435 *Quantum Computation*, lecture notes (2022), especially Lectures 3, 19–25. `https://math.mit.edu/~shor/435-LN/`. Selected PDFs opened in the web reader; local downloads failed.

[3] J. Preskill, Caltech Ph219/CS219 lecture notes, Chapters 2 (2015), 3 (2015), 7 (2026). `https://www.preskill.caltech.edu/ph229/`. Selected PDFs downloaded; the Chapter 7 PDF has 92 pages.

[4] D-Wave, *QUBOs and Ising Models*; *Reformulating a Problem*; *What is Quantum Annealing?* Model guide; reformulation; annealing. Training catalogue inspected; account-only courses not used.

[5] PennyLane, *Codebook: Variational Quantum Algorithms*; *param_shift* documentation. Codebook; gradient API. Interactive challenge content was not fully available to the reader.

[6] IBM Quantum Learning, *Basics of Quantum Information*, *Fundamentals of Quantum Algorithms*, *Variational Algorithm Design*. `https://quantum.cloud.ibm.com/learning`. Foundational courses cross-checked through [1]; variational workflow inspected separately.

[7] TU Delft OCW, *Quantum Cryptography*, Week 6: Quantum Key Distribution Protocols, Sections 6.1–6.3. Official written notes. BB84, adversary models and authentication.

[8] L. K. Grover, *A fast quantum mechanical algorithm for database search* (1996). arXiv:quant-ph/9605043. Original search paper; derivation cross-checked with [1,2].

[9] G. Brassard, P. Høyer, M. Mosca and A. Tapp, *Quantum Amplitude Amplification and Estimation* (2000). arXiv:quant-ph/0005055. Generalisation and success-scaling assumptions.

[10] E. Farhi, J. Goldstone and S. Gutmann, *A Quantum Approximate Optimization Algorithm* (2014). arXiv:1411.4028. Alternating circuit construction, not evidence for Athena advantage.

[11] M. Schuld et al., *Evaluating analytic gradients on quantum hardware*, Phys. Rev. A 99, 032331 (2019). arXiv:1811.11184. Gradient rules; [5] gives implementation conventions.

[12] J. R. McClean et al., *Barren plateaus in quantum neural network training landscapes*, Nature Communications 9, 4812 (2018). arXiv:1803.11173. Circuit-dependent concentration result.

[13] NIST, *Approval of FIPS 203, 204 and 205* (13 August 2024); *Migration to PQC FAQ*. Standards announcement; `https://pages.nist.gov/nccoe-migration-post-quantum-cryptography/`. Recheck deployment guidance when implementing.